

Proyecto Estratégico “Madurez, Adaptabilidad, Resiliencia y Continuidad en Organizaciones Digitalizadas. MARC.0 Diligencia)”

Informe de seguimiento

(Periodo octubre 2024 – septiembre 2025)

Introducción.....	4
Objetivos.....	4
Desarrollo del Proyecto	5
Investigación	5
OpenUI (GitHub).....	5
Towards an AI co-scientis	5
Construcción de un grafo de conocimiento con LlamaCloud y Neo4J.....	5
LangChain y LlamaIndex.....	7
N8N.....	7
MCP	8
A2A y MoE	8
Formación.....	9
Diseño	9
Extracción de datos	10
Persistencia	11
LLM y lógica de negocio.....	11
Aplicación web.....	11
Frontend.....	15
Implementación	16
Extracción de datos	16
Base de datos orientada a grafos	16
Base de datos SQL	17
Variables Personalizadas.....	20
Aplicación web.....	20
Chats y Conversaciones	23
Lógica de negocio	24
Gestión del Proyecto.....	26
Actas de reunión	26
21 de marzo de 2025.....	27
27 de marzo de 2025.....	28
28 de marzo de 2025.....	29
10 de abril de 2025.....	30
2 de mayo de 2025.....	31
8 de mayo de 2025.....	33
17 de junio de 2025	34
Otros documentos	35

Especificación de Requisitos.....	35
Arquitectura del Sistema	39
Especificación de Requisitos de WikIA.....	43
Especificación de la API REST	45
Investigación y evaluación del uso de MCPs para web scraping con Puppeteer y Filesystem.....	49
Participantes.....	51
Conclusión	52
Situación actual.....	52

Introducción

En un entorno cada vez más digitalizado, la ciberseguridad se ha convertido en una prioridad para gobiernos, empresas y ciudadanos. El constante aumento de amenazas digitales y la creciente sofisticación de los ciberataques requieren soluciones tecnológicas capaces de prevenir, detectar y responder de manera eficiente. En este contexto, surge el proyecto MARCO Digiliencia, una iniciativa desarrollada por estudiantes de la Universidad de León, con el objetivo de ofrecer una herramienta moderna, escalable y segura que fortalezca la protección digital de los ciudadanos.

El proyecto está enfocado en dar soporte al servicio 017, una línea de ayuda en ciberseguridad que atiende incidencias, proporciona formación y ofrece retroalimentación a los usuarios. Dado el volumen de consultas y la necesidad de atención especializada en distintos niveles (N1, N2 y N3), se propone la creación de un chatbot de ciberseguridad, potenciado por inteligencia artificial y respaldado por una base de datos orientada a grafos. Esta solución busca facilitar el acceso a información confiable, ofrecer respuestas automáticas a problemas comunes y derivar casos complejos a agentes humanos cuando sea necesario.

La inteligencia artificial, entrenada, validada y optimizada mediante técnicas como el ajuste de hiperparámetros y validación cruzada, permite lograr una alta precisión en la detección de incidencias y la recomendación de soluciones. Este sistema no solo está pensado para actuar ante incidentes ya ocurridos, sino también para prevenir ataques, educar al usuario y fomentar buenas prácticas en ciberseguridad.

En un momento en que los ciberataques representan ya el 1.5% del PIB mundial, herramientas como la que plantea MARCO Digiliencia son fundamentales para reducir el impacto económico y social de estas amenazas. La finalidad es clara: proteger, informar y capacitar al ciudadano mediante el uso responsable e innovador de la tecnología.

Objetivos

El proyecto tiene como objetivo principal desarrollar un servicio accesible y adaptable a las características de sus usuarios, considerando tanto su nivel técnico como su contexto social. Se busca crear una solución escalable, sostenible y de bajo coste, que permita su mantenimiento a largo plazo sin comprometer la calidad del servicio.

Asimismo, se pretende garantizar que la información proporcionada sea actual, fiable y basada en fuentes verificadas, asegurando la trazabilidad de las respuestas. El proyecto también contempla el cumplimiento de principios éticos, de seguridad y de la normativa vigente, especialmente en lo referente a la protección de datos y la privacidad de los usuarios.

Otro de los objetivos es facilitar la interacción mediante interfaces intuitivas y documentación clara, además de implementar un sistema de gestión de usuarios que permita diferenciar perfiles según sus necesidades. Finalmente, se busca ofrecer a cada usuario acceso a su historial de uso, contribuyendo a una experiencia más personalizada y continua.

Desarrollo del Proyecto

El proyecto se ha llevado a cabo en varias fases bien definidas. En la primera fase, se realizó un proceso de investigación para identificar las tecnologías más adecuadas y comprender su propósito dentro del sistema. Paralelamente, se llevaron a cabo formaciones en diversas herramientas con el objetivo de adquirir un conocimiento técnico básico que facilitara su posterior integración.

Durante la segunda fase, se definieron las tecnologías que formarían parte del desarrollo y se estableció la metodología ágil que se seguiría a lo largo del proyecto. Además, se configuró el repositorio remoto y se realizó la primera carga de archivos, sentando así las bases para el trabajo colaborativo y el control de versiones.

En la tercera y última fase, se procedió al desarrollo activo del sistema. Esto incluyó la incorporación progresiva de código, la recolección y procesamiento de información a través de técnicas de *scraping*, y su posterior inserción en la base de datos orientada a grafos Neo4j. Asimismo, se integraron herramientas basadas en inteligencia artificial con el fin de mejorar la gestión y precisión de la información procesada.

Para más información, puede solicitar acceso a los recursos del proyecto¹.

Investigación

Durante el proyecto se han estudiado varias herramientas y artículos científicos.

OpenUI (GitHub)

Esta herramienta fue explorada como posible interfaz de usuario modular y gratuita para la interacción con sistemas complejos. Aunque se valoró su potencial para ofrecer un front-end abierto y reutilizable, quedó solo como propuesta y no fue incorporada al proyecto.

Towards an AI co-scientist

El artículo presenta el AI co-scientist, un sistema multi-agente basado en Gemini 2.0 que asiste en el descubrimiento científico generando hipótesis originales y verificables mediante un ciclo de generación, debate y evolución. Opera bajo un paradigma "scientist-in-the-loop", donde científicos revisan y validan las propuestas fundamentadas en literatura. Validado en biomédica, identificó candidatos para leucemia, nuevas dianas terapéuticas y mecanismos evolutivos, superando a otros LLM en calidad de hipótesis y evidencia experimental.

Construcción de un grafo de conocimiento con LlamaCloud y Neo4J

Contexto y motivación

La técnica de Retrieval-Augmented Generation (RAG) mejora significativamente las capacidades de los modelos de lenguaje al complementarlos con fuentes externas de conocimiento. Sin embargo, la búsqueda por similitud semántica convencional puede fallar al capturar relaciones complejas y contextos distribuidos en múltiples documentos. Transformar documentos no estructurados en grafos de conocimiento permite realizar consultas relacionales más ricas, razonamiento contextual y travesías complejas que superan las limitaciones de la búsqueda vectorial simple.

¹ www.github.com/Digiliencia/InfraestructuraDigiliencia

Las herramientas principales empleadas son **LlamaCloud**, que integra herramientas como **LlamaClassify**, que clasifica documentos (por ejemplo, contratos legales), y **LlamaExtract**, que extrae atributos relevantes según el tipo de documento identificado, y **Neo4j**, utilizado como plataforma para almacenar y representar el grafo de conocimiento resultante.

Flujo de trabajo (pipeline)

El ejemplo guía a través de un flujo completo aplicado al procesamiento de documentos legales, con las siguientes etapas:

1. **Instalación y configuración:** se emplean librerías como llama-index-workflows, llama-cloud-services, jsonschema, openai, neo4j y llama-index-lms-openai.
2. **Clasificación de documentos** con LlamaClassify para determinar la categoría de cada contrato.
3. **Extracción de información** usando LlamaExtract, según los atributos relevantes a cada tipo de contrato.
4. **Almacenamiento en Neo4j:** transformación de la información estructurada en nodos y relaciones dentro de un grafo de conocimiento.

Aplicación y beneficios

- **Modelado relacional:** los grafos permiten visualizar y analizar cómo los distintos elementos del documento se relacionan, lo que posibilita consultas más sofisticadas mediante técnicas como GraphRAG.
- **Dominio legal como caso de uso:** en los documentos legales, donde las relaciones entre cláusulas, referencias y entidades son esenciales, esta aproximación mejora notablemente la precisión y contextualización.
- **Ventajas técnicas:** Permite consultas por traversals, multi-hop y contexto más profundo. Además, facilita el razonamiento estructurado más allá de similitud superficial.

La construcción de un grafo de conocimiento a partir de documentos no estructurados mediante el uso combinado de LlamaCloud y Neo4j representa un avance significativo en la integración de inteligencia artificial y bases de datos orientadas a grafos. Al superar las limitaciones de la búsqueda semántica tradicional, esta solución permite consultas más complejas, razonamiento contextual profundo y una representación estructurada del conocimiento.

Particularmente en el dominio legal, donde las relaciones entre cláusulas, entidades y referencias son críticas, este enfoque aporta una mayor precisión en la interpretación y recuperación de información. El uso de herramientas como LlamaClassify y LlamaExtract automatiza el procesamiento documental, mientras que Neo4j facilita la persistencia y navegación eficiente del conocimiento extraído.

En conjunto, este pipeline no solo mejora la calidad de los sistemas RAG mediante técnicas como GraphRAG, sino que sienta las bases para desarrollos más avanzados en comprensión de documentos, generación de respuestas enriquecidas y soporte a la toma de decisiones informadas en dominios complejos.

LangChain y LlamaIndex

Para seleccionar el framework que servirá como interfaz entre el modelo de lenguaje grande (LLM) y la base de datos, se consideraron las dos principales alternativas disponibles en el mercado: LangChain y LlamaIndex.

LangChain constituye un framework de propósito general, no especializado en ningún ámbito particular. Se caracteriza por ofrecer una amplia gama de herramientas aplicables a diversos componentes de un sistema de inteligencia artificial (desde el LLM hasta agentes o gestión de memoria, entre otros). Dispone de conectores para el manejo adecuado de los datos, aunque su principal ventaja frente a LlamaIndex radica en su flexibilidad, un aspecto que podría resultar valioso para gestionar la complejidad integral del proyecto, dada la diversidad de ramas y casos de uso que se están desarrollando.

Por su parte, LlamaIndex es también un framework orientado al trabajo con LLMs, pero con una diferencia fundamental: se trata de una solución especializada. Está específicamente diseñado para el consumo y consulta de datos, y, de hecho, opera sobre LangChain en su capa subyacente. Ofrece una amplia variedad de funciones preconstruidas que facilitan notablemente la implementación. Además, está concebido para proporcionar información a partir de datos previamente suministrados, lo que lo convierte en una opción idónea para nuestro caso de uso.

Una analogía apropiada en términos de lenguajes de programación sería comparar Java, un lenguaje de alto nivel, con C, uno más versátil y que permite un control más granular.

En virtud de su mayor simplicidad de implementación, su enfoque específico en la presentación de información a partir de datos almacenados y la reducción en la cantidad de instrucciones requeridas para su correcto funcionamiento, se ha seleccionado LlamaIndex como el framework para este proyecto.

N8N

Se estudió el uso de N8N en el proyecto, una plataforma de automatización de flujos de trabajo basada en bajo código. El término "bajo código" hace referencia a un enfoque de desarrollo que minimiza la necesidad de escribir código manualmente, utilizando herramientas visuales, plantillas y componentes predefinidos para construir aplicaciones o procesos de manera más rápida y accesible. Cada nodo representa una acción concreta, como enviar un correo, escribir en una base de datos o ejecutar un script personalizado, lo que permite encadenar tareas de forma automatizada.

Durante el análisis se realizaron pruebas exhaustivas de sus capacidades aplicables al proyecto. Se desarrolló un prototipo en el que un modelo de OpenAI interactuaba con Google Calendar a través de una API, proporcionando información sobre los eventos del usuario siguiendo patrones predefinidos. Además, se exploró la ejecución local de proyectos mediante Docker sin necesidad de licencias, así como la posibilidad de desplegar proyectos en la nube para trabajo colaborativo en tiempo real.

No obstante, durante la evaluación se identificaron limitaciones significativas. El enfoque de bajo código, aunque simplifica la adopción, reduce la versatilidad, ya que ciertas operaciones avanzadas requieren soluciones menos directas o no son posibles dentro del entorno visual. Asimismo, depender de la interfaz gráfica limita la potencia y eficiencia frente a alternativas basadas en CLI, donde la programación directa ofrece mayor control, flexibilidad y capacidad de integración con entornos complejos.

Respecto a su implementación concreta en el proyecto, N8N era técnicamente viable. Sin embargo, el modelo de licencias supuso un obstáculo crítico: aunque es de código abierto y permite uso privado, exige la adquisición de una licencia para cualquier fin comercial, incluyendo proyectos sin ánimo de lucro destinados a ofrecer servicios públicos. En consecuencia, se decidió relegar su uso al desarrollo de herramientas internas que no requieran licencia comercial, aprovechando su facilidad de uso sin comprometer la viabilidad del proyecto.

MCP

El Model Context Protocol (MCP) es un estándar abierto de protocolos que permite a las aplicaciones de inteligencia artificial interactuar de manera dinámica con herramientas y fuentes de información externas al sistema. Su principal objetivo es facilitar el acceso seguro y organizado a información en tiempo real y contexto específico, mejorando así las capacidades de los modelos de lenguaje.

Entre las ventajas principales asociadas al uso de MCP se encuentran la capacidad de razonamiento basado en datos actualizados, lo que reduce significativamente la posibilidad de alucinaciones y errores asociados con información desactualizada, así como su escalabilidad.

No obstante, también presenta aspectos negativos. Aunque MCP fue diseñado considerando aspectos de seguridad, su implementación implica conectar el modelo de lenguaje (LLM) a servidores externos, lo que introduce puntos adicionales de acceso y posibles vectores de vulnerabilidad si no se configura adecuadamente. Adicionalmente, actualmente cuenta con un sistema de gestión de permisos limitado, cuyo control granular aún se encuentra en desarrollo.

Asimismo, al tratarse de una tecnología emergente, cuenta con un ecosistema aún incipiente, lo que implica una trayectoria de evolución incierta y dificulta prever su adopción y mejora continua.

Considerando los riesgos potenciales en materia de seguridad y privacidad, se ha determinado no adoptar esta tecnología en el estado actual de su desarrollo.

A2A y MoE

En el estudio de servicios para el proyecto se analizó el paradigma Agent-to-Agent (A2A), que se basa en la interoperabilidad entre agentes de software autónomos y heterogéneos mediante protocolos de comunicación estandarizados. Esta arquitectura permite que distintos sistemas —como APIs, servicios de computación o modelos de lenguaje— se descubran, negocien tareas y coordinen acciones de manera dinámica. Desde la perspectiva del proyecto, A2A ofrece un alto potencial de modularidad y escalabilidad, ya que permite integrar agentes independientes sin modificar la arquitectura global, e incluso combinar LLM de diferentes proveedores. Además, la comunicación basada en APIs o protocolos de mensajería facilita la extensión de funcionalidades y la integración de nuevos agentes en el futuro.

Sin embargo, la implementación de un sistema A2A presenta limitaciones relevantes. Su complejidad de coordinación requiere un diseño cuidadoso y protocolos robustos, y la comunicación externa entre agentes es más lenta que la comunicación interna de un único sistema. En el contexto del proyecto, que no se centraliza sobre plataformas como n8n, establecer un sistema multi-agente resultaría complicado y costoso en términos de desarrollo.

Por estas razones, se considera que A2A es un enfoque interesante a largo plazo, pero no viable ni prioritario en la fase actual del proyecto.

En paralelo, se evaluó la arquitectura Mixture of Experts (MoE) utilizada en modelos de lenguaje avanzados como DeepSeek. MoE distribuye la carga computacional entre múltiples subredes especializadas o "expertos", activando únicamente aquellos necesarios para cada tarea mediante un gating network. Esta técnica permite optimizar la eficiencia interna del modelo y su escalabilidad, sin incrementar excesivamente los costos de inferencia. En el proyecto, la arquitectura MoE se adapta bien a la tecnología del LLM empleado, garantizando un rendimiento estable y sin problemas de procesamiento. No obstante, su integración aumenta la complejidad del diseño y la gestión del modelo, y los expertos internos no ofrecen la flexibilidad de agentes independientes como en A2A.

En conclusión, la evaluación indica que la mejor estrategia para la fase actual es centrarse en un diseño basado en un único agente. A2A y MoE se mantienen como alternativas a considerar en el futuro: A2A si se requiere modularidad y colaboración entre múltiples sistemas, y MoE si se necesita optimizar la eficiencia interna del modelo. Esta aproximación permite mantener el proyecto viable y controlable, sin comprometer su rendimiento ni su escalabilidad a corto plazo.

Formación

El equipo del proyecto ha completado varias formaciones de las tecnologías a usar en el proyecto. Entre las plataformas empleadas destacan Neo4j Academy y DeepLearning. A parte de realizar varias investigaciones sobre las tecnologías a usar en el proyecto como podrían ser N8N, MCP, A2A y sobre varias publicaciones académicas. Además, parte del equipo acudió a la formación del Cybersecurity Summer Bootcamp.

Diseño

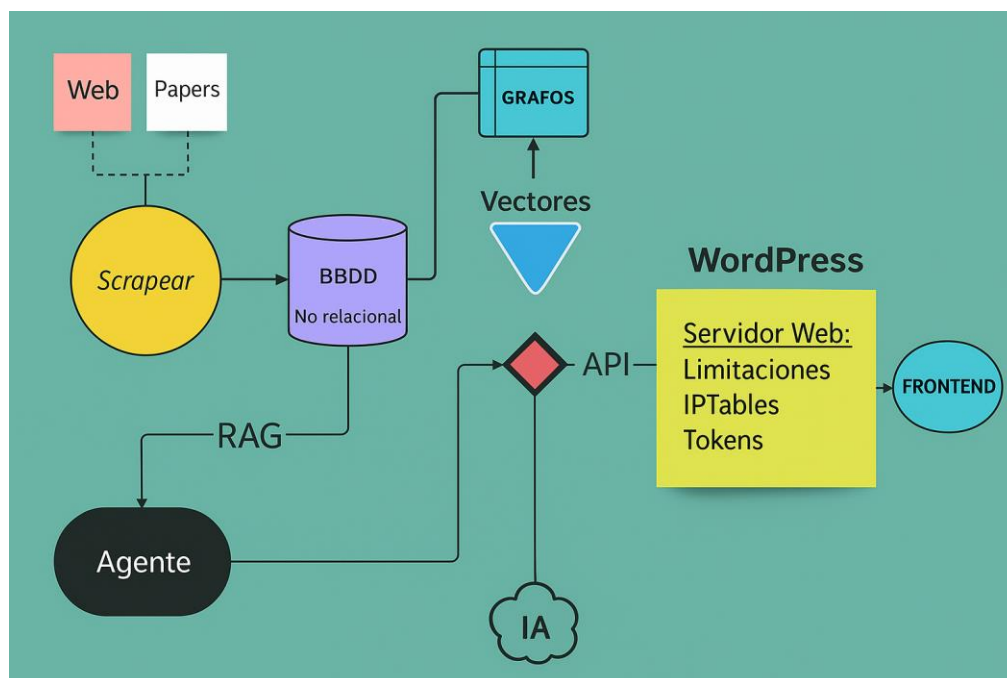
La estructura del proyecto sigue un esquema en tres capas (capas de presentación, lógica de negocio y persistencia) y está compuesta por módulos aislados y sigue un esquema cliente-servidor.

La arquitectura cliente-servidor es la más utilizada en servicios web, ya que su esquema es ideal para este tipo de servicios. Además, la estructura típica de estos servicios presenta una clara distinción entre la presentación (*frontend*) y la lógica de negocio y persistencia (*backend*) que se refleja en el uso del modelo en tres capas.

El diseño de la arquitectura sigue los principios generales del proyecto: soberanía de los datos e independencia de terceros. En consecuencia, se ha buscado un diseño que mantenga todos los servicios ejecutándose en servidores propios, minimizando la dependencia de proveedores externos, tanto a nivel de *software* como de *hardware*.

Dentro de cada capa es también posible distinguir diferentes servicios o funcionalidades más o menos aisladas que podemos tratar como microservicios (aunque la denominación de micro no guarda relación con el tamaño o coste del mismo).

El proyecto consta de cinco módulos principales: *extracción de datos*, persistencia (bases de datos SQL y orientada a grafos), LLM y lógica de negocio, servicio web y *frontend*. El lado del cliente (*frontend*) será una interfaz web que gestionará la interacción del usuario con el servicio, alojado en el servidor. El servicio web implementará una interfaz API que constituye el canal de comunicación entre el cliente y el servidor.



A continuación, se presentan los diferentes módulos.

Extracción de datos

Se han desarrollado ocho programas automatizados de rastreo web destinadas a ocho sitios web para la extracción de los datos, organizados en módulos funcionales. A continuación, se describen dichos módulos:

Los programas automatizados de rastreo web desarrollados para la extracción de datos han implementado específicamente a los siguientes sitios webs:

- INCIBE (Instituto Nacional de Ciberseguridad)
- CERT-CNI (Centro Nacional de Inteligencia)
- NCSC (National Cyber Security Center)
- Weforum (World Economic Forum)
- Candian Center of Ciber Security
- NIST (National Institute of Stanford and Technology)
- CISA (America's CyberDefense Agency)

La mayoría de los programas de rastreo web desarrollados presentan una estructura de implementación semejante, fundamentada en el uso de las siguientes funcionalidades principales:

La primera funcionalidad rastrea artículos, noticias, publicaciones académicas, eventos o conferencias para luego la extracción de sus datos.

La segunda funcionalidad se encarga de rastrear las diferentes secciones de los sitios web.

La configuración de los distintos programas automatizados ha sido diseñada para permitir su ejecución concurrente, basada en el parámetro configurable correspondiente al número de días a extraer. Esta arquitectura facilita una ejecución dinámica y adaptable, permitiendo escalar o ajustar el proceso de extracción de datos según los requerimientos específicos del entorno o del caso de uso.

Persistencia

El dataverso es la colección de datos, información, que el modelo de inteligencia artificial empleará como fuente de conocimiento para generar sus respuestas. Es el corazón de proyecto y fundamental en la calidad del servicio.

Es necesario recabar un volumen ingente de datos para generar un dataverso adecuado. En consecuencia, se ha decidido emplear una base de datos orientada a grafos. Este tipo de bases de datos son eficientes en el manejo del tipo de datos de los que consta el dataverso.

Por otro lado, se emplea una base de datos SQL tradicional para el manejo del resto de información necesaria, principalmente relacionada con la prestación del servicio, como los usuarios registrados, el historial de conversaciones o los registros del sistema (*logs*).

LLM y lógica de negocio

Los agentes son quienes procesarán las peticiones de los usuarios y generará las respuestas haciendo uso del dataverso. La lógica de negocio provee al agente de herramientas para realizar su cometido, como el acceso al dataverso (acceso a la base de datos orientada a grafos), al historial de conversación del usuario, acceso a servicios en internet o herramientas generativas (como generación de imágenes).

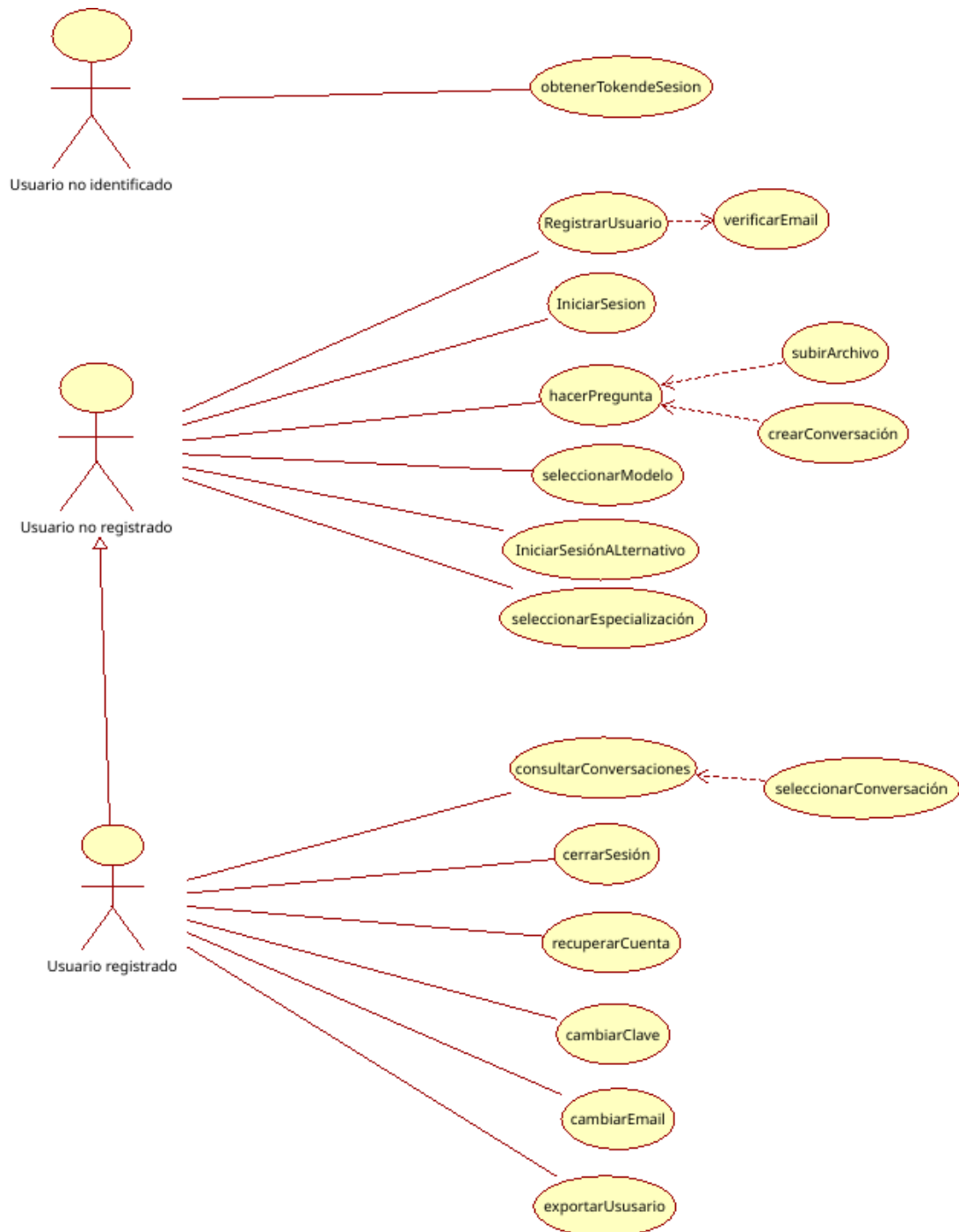
Aplicación web

La aplicación web realizará la gestión de usuarios y sesiones, delegando la generación de respuestas a la lógica de negocio. Es la encargada de servir y gestionar la interacción entre los usuarios y el sistema.

La comunicación entre el cliente y el servidor se realizará principalmente mediante una API. Ésta constituye la interfaz de interacción del *frontend* con la lógica de negocio. Es la pasarela por la que el usuario se comunicará con el modelo a través de la interfaz de usuario.

El diseño de la API responde al tipo REST. Su uso como interfaz entre el cliente y el servidor es una forma simple y eficiente de comunicarse ampliamente utilizada en el ámbito de los servicios web y que mejora protocolos anteriores como SOAP, verboso e ineficiente. Además, al ser un protocolo estándar y sencillo es más fácil de implementar, mantener y proteger. También reduce el acoplamiento entre ambas capas y facilita la futura adición de nuevas interfaces de usuario. Dese el punto de vista de la seguridad se reduce la superficie de ataque.

Los casos de uso (CdU) de la aplicación web se muestran en la imagen y se detallan a continuación.



Los actores (tipos de usuario) son los siguientes:

- **Usuario no autenticado:** Un usuario que ha accedido a la aplicación, pero no posee un token de sesión válido. Este estado es el predeterminado para cualquier visitante nuevo.
- **Usuario no registrado:** Un usuario que no ha creado una cuenta en el sistema, pero que ha obtenido un **token de sesión temporal**. Este estado le permite interactuar de forma limitada con algunas funcionalidades.
- **Usuario registrado:** Un usuario que ha completado el proceso de registro y ha iniciado sesión. Posee una cuenta en la base de datos y un **token de sesión válido** que le otorga acceso completo.

Las acciones definidas son las siguientes:

a) Obtener Token

- a. **Precondición:** El usuario no tiene un token de sesión activo.
- b. **Postcondición:** El usuario recibe un token de sesión, lo que cambia su estado a "Usuario no registrado".
- c. **Descripción:** Este proceso se ejecuta automáticamente cuando un usuario accede por primera vez a la aplicación sin un token. El sistema genera y asigna un identificador único (el token) que le permite iniciar una conversación sin necesidad de registrarse inmediatamente.

b) Registrar Usuario

- a. **Precondición:** El usuario se encuentra en el estado de "Usuario no registrado".
- b. **Postcondición:** El usuario se registra en la base de datos y su estado cambia a "Usuario registrado", iniciando automáticamente la sesión.
- c. **Excepción:** El proceso fallará si el correo electrónico proporcionado ya está en uso o si el usuario no verifica su dirección de correo a través del enlace enviado.
- d. **Descripción:** El usuario proporciona un correo electrónico y una contraseña para crear su cuenta. El sistema valida que el correo no esté asociado a otra cuenta, lo registra en la base de datos y envía un correo de verificación. Al hacer clic en el enlace, el usuario completa el registro y se autentica en la aplicación.

c) Iniciar Sesión

- a. **Precondición:** El usuario no está autenticado (es "no registrado" o "no autenticado").
- b. **Postcondición:** El usuario se autentica y pasa al estado de "Usuario registrado".
- c. **Excepción:** El acceso será denegado si las credenciales (correo/contraseña) son incorrectas o si falla la verificación del CAPTCHA.
- d. **Descripción:** El usuario introduce su correo y contraseña. El sistema verifica la combinación de credenciales y la validación del CAPTCHA. Si todo es correcto, le asigna un nuevo token de sesión que le concede acceso como usuario registrado.

d) Hacer Pregunta

- a. **Precondición:** El usuario debe estar autenticado.
- b. **Postcondición:** El usuario recibe una respuesta de la IA. La pregunta y la respuesta se guardan en la base de datos.
- c. **Descripción:** El usuario envía una pregunta en formato de texto. El sistema la procesa a través del modelo de lenguaje configurado, genera una respuesta y la muestra al usuario, almacenando la interacción completa en la conversación activa.

e) Recuperar Cuenta

- a. **Precondición:** El usuario no está autenticado.
- b. **Postcondición:** El usuario recibe un enlace o código temporal para restablecer su contraseña.
- c. **Excepción:** Si el correo electrónico no está registrado en el sistema, no se enviará ningún mensaje.

- d. **Descripción:** El usuario introduce el correo de su cuenta. El sistema verifica que el correo existe y, de ser así, le envía un enlace seguro que le permite crear una nueva contraseña y recuperar el acceso a su cuenta.

f) Cerrar Sesión

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** El token de sesión del usuario es invalidado, y su estado cambia a "Usuario no autenticado".
- c. **Excepción:** El proceso no se puede completar si el token de sesión ya ha expirado o no es válido.
- d. **Descripción:** El usuario solicita cerrar su sesión. El sistema invalida el token de sesión asociado a su cuenta, eliminando su autenticación y obligándolo a volver a iniciar sesión para acceder a las funcionalidades de usuario registrado.

g) Cambiar Contraseña

- a. **Precondición:** El usuario debe estar autenticado.
- b. **Postcondición:** La contraseña del usuario se actualiza en la base de datos.
- c. **Excepción:** El cambio fallará si la contraseña actual introducida no coincide con la almacenada.
- d. **Descripción:** El usuario proporciona su contraseña actual y la nueva que desea usar. El sistema valida la contraseña actual y, si es correcta, actualiza la base de datos con la nueva. Por seguridad, el token de sesión actual se invalida y el usuario debe volver a iniciar sesión con su nueva contraseña.

h) Cambiar Email

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** La dirección de correo electrónico del usuario se modifica en la base de datos.
- c. **Excepción:** El cambio fallará si la contraseña actual es incorrecta o si el nuevo correo electrónico ya está en uso.
- d. **Descripción:** Similar al cambio de contraseña, el usuario debe introducir su contraseña actual y la nueva dirección de correo. El sistema envía un correo de verificación a la nueva dirección. Solo después de la confirmación, se actualiza la base de datos y se invalida el token de sesión.

i) Consultar Conversaciones

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** El sistema devuelve una lista con los títulos de todas las conversaciones del usuario.
- c. **Excepción:** Si el usuario no ha iniciado ninguna conversación, se devolverá una lista vacía.
- d. **Descripción:** El usuario solicita un historial de sus chats. El sistema consulta la base de datos y le proporciona una lista de los títulos de todas las conversaciones que ha tenido, permitiéndole navegar por ellas.

j) Verificar Email

- a. **Precondición:** El usuario ha recibido un correo de verificación.
- b. **Postcondición:** El correo electrónico se marca como verificado y la cuenta se activa por completo.
- c. **Excepción:** El proceso no se completa si el enlace ya ha sido utilizado o ha expirado.

- d. **Descripción:** Al hacer clic en el enlace del correo de verificación, el sistema valida el token del enlace. Si es válido, la cuenta del usuario se activa, otorgándole acceso a todas las funcionalidades.

k) Seleccionar Conversación

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** El usuario recibe el contenido de la conversación seleccionada. A partir de ese momento, las nuevas preguntas se añadirán a esta conversación.
- c. **Excepción:** Se devolverá un error si el usuario intenta acceder a una conversación que no existe o que no le pertenece.
- d. **Descripción:** El usuario selecciona una conversación de su lista de historial. El sistema recupera y muestra el contenido completo de la misma.

l) Seleccionar Modelo

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** El modelo de IA predeterminado para las nuevas preguntas cambia.
- c. **Excepción:** El acceso a ciertos modelos puede estar restringido para usuarios no registrados.
- d. **Descripción:** El usuario puede elegir entre diferentes modelos de IA disponibles. Una vez seleccionado, todas las preguntas subsiguientes se procesarán con el modelo elegido hasta que se cambie de nuevo.

m) Seleccionar Especialización

- a. **Precondición:** El usuario está autenticado.
- b. **Postcondición:** Se crea una nueva conversación con una especialización temática definida.
- c. **Excepción:** El acceso a ciertas especializaciones puede estar restringido para usuarios no registrados.
- d. **Descripción:** El usuario elige un tema o "especialización" para una nueva conversación. Al seleccionar una especialización, se crea un nuevo chat dedicado a ese tema, y esta no puede ser modificada dentro de esa conversación.

n) Crear Conversación

- a. **Precondición:** El usuario está autenticado o en el estado "no registrado".
- b. **Postcondición:** Se crea una nueva entrada de conversación en la base de datos.
- c. **Excepción:** Si el usuario "no registrado" crea una nueva conversación, su conversación anterior será eliminada para evitar sobrecarga en la base de datos.
- d. **Descripción:** Se genera una nueva conversación, y su título se crea automáticamente basándose en el contenido de la primera pregunta del usuario.

Frontend

Este módulo es la cara visible del servicio para el usuario. Inicialmente será una interfaz web siguiendo la estructura típica de los servicios de IA conversacional (*chatbot*). Por ahora no se ha abordado su diseño.

Implementación

Esta sección presenta la implementación concreta realizada y muestra, de manera pormenorizada, el trabajo realizado hasta el momento.

Extracción de datos

Se evaluaron distintas tecnologías para el desarrollo del webscrapping, entre ellas fueron Playwright, BeautifulSoup, Puppeteer y Selenium. Después de un análisis exhaustivo, y pruebas prácticas realizadas a cada una de estas tecnologías se optó por la tecnología de Selenium, ya que permite controlar un navegador web como si fuera un usuario. Además de contar con una amplia comunidad activa que contribuye continuamente al crecimiento de sus funcionalidades y a la expansión de su documentación, Selenium es una tecnología multiplataforma.

Base de datos orientada a grafos

Se valoraron distintos tipos de bases de datos, tanto relacionales como documentales, orientadas a grafos y vectoriales. Tras un análisis detallado, y después de analizar gestores de bases de datos concretos, se optó por Neo4j, un sistema gestor de base de datos orientada a grafos. Se tomó esta decisión dada la ingente cantidad de cursos que esta plataforma ofrece gratuitamente para aprender a utilizar la plataforma, las bases de datos vectoriales y sus consultas, integraciones con lenguajes de programación directamente y con frameworks especializados, y también ejemplos prácticos de cómo crear sistemas RAG a través de bots conversacionales.

El diseño de la BBDD se ha pensado de manera escalable, pues para crear un dataverso se debe comenzar por elementos concretos más tangibles. Se tomó la decisión de realizar web scrapping. Gracias a ello comenzaron a emerger las primeras entidades (nodos) y sus relaciones:

- **Noticia:** nodo que contiene una noticia, y su contenido. Tiene además el titular, la dirección URL y fecha de publicación.
- **Autor:** nodo que es también de tipo Persona. Contiene el nombre. Puede contener un correo electrónico y está relacionado con la noticia.
- **Agencia de Noticias:** es de tipo Organización. Contiene el nombre y está relacionado con las noticias. Esto permite saber de manera rápida quién publica más noticias de ciertas temáticas y a quién pertenece cada noticia.
- **Tópicos:** son conceptos relacionados con el mundo de la ciberseguridad (por ejemplo, vulnerabilidad, phishing, etc). Se obtuvieron de la página del INCIBE junto con sus definiciones.
- **Campos relacionados:** sectores del mundo real con el que las noticias pueden estar relacionadas. Por ejemplo, agricultura, turismo, finanzas o comercio.
- **Chunk:** contiene un fragmento de una noticia y su representación vectorial (embedding). Se emplea para buscar por contenido o similitud. Se ha necesitado fraccionar la noticia en varios chunks debido a la extensión de los contenidos de algunas de ellas, sabiendo que actualmente los embeddings pierden exactitud al representar más de 400 caracteres. Se desarrolló un sistema de IA que permite asociar noticias automáticamente a tópicos y a campos relacionados, creando autónomamente las relaciones correspondientes. De este modo se puede consultar noticias relacionadas con ciertos campos.

Se ha dejado la BBDD preparada para poder ampliar (además de noticias) con papers, cursos, o cualquier otro campo que sea de interés. Esta es una de las grandes ventajas de

las bases de datos no relacionadas, que no precisan de la creación de un esquema, lo que ofrece mayor flexibilidad.

Actualmente se está ejecutando la aplicación en el servidor web de Neo4j Aura DB.

Respecto a los puntos anteriores, se recomienda ver el repositorio de GitHub² donde está el código fuente desarrollado hasta el momento:

Base de datos SQL

Se ha empleado la base de datos SQL PostgreSQL. Esta base de datos posee una licencia libre y, por tanto, compatible con el objetivo de independencia tecnológica. Además, es una base de datos ampliamente utilizada, con una gran comunidad y apta para grandes volúmenes de datos, por lo que es adecuada para emplearse en el proyecto. Su dialecto SQL no difiere demasiado del estándar SQL por lo que reduce el acoplamiento a esta base de datos concreta. Esto facilita una posible migración del proyecto a otra base de datos.

Para generar la base de datos se han empleado varios *scripts* y otros ficheros con el objetivo de automatizar la puesta en marcha de la base de datos. De este modo, una nueva instancia de la base de datos es generada de forma inmediata.

Los ficheros de configuración incluyen *scripts* SQL para la inicialización de la base de datos, la creación del esquema, vistas y procedimientos, así como un completo conjunto de pruebas para verificar la estructura, permisos y funcionalidad de la base de datos. Esto último es especialmente relevante dado el campo de trabajo del proyecto, así como la sensibilidad de los datos que los usuarios pueden introducir en la aplicación.

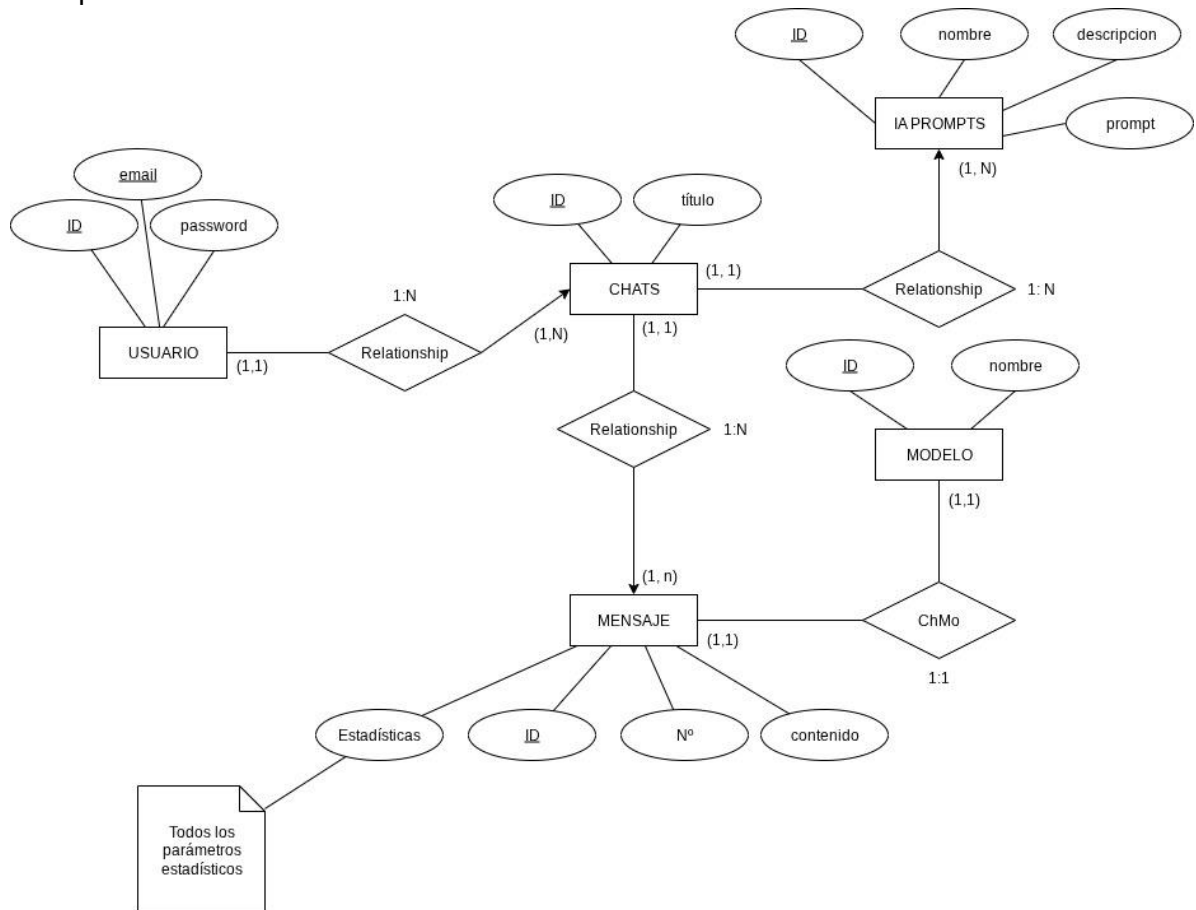
La base de datos se inicializa y configura usando un único script de *shell*. El *script* leerá las variables de entorno, procesará las plantillas SQL y ejecutará todos los scripts en el orden correcto para dejar la base de datos lista para su uso. La configuración de las variables de entorno se realiza en el archivo `.env`, en el directorio raíz del proyecto. Este fichero contiene los parámetros de configuración, así como todas las credenciales necesarias de los usuarios de la base de datos (no usuarios finales).

La estructura de archivos es la siguiente:

- `execute_scripts.sh`: Orquesta todo el proceso de inicialización de la base de datos dentro de PostgreSQL.
- `scripts/`: Contiene los archivos SQL.
 - `01-init-db.sql.template`: Crea la base de datos y los roles de usuario.
 - `02-create-schema.sql`: Define y crea todas las tablas.
 - `03-views.sql`: Crea las vistas de la base de datos.
 - `04-procediments.sql`: Crea funciones y procedimientos almacenados.
 - `10-grant-permissions.sql.template`: Asigna permisos específicos a cada rol.
 - `test/`: Contiene las pruebas `pytest`.
 - `conftest.py`: Prepara el entorno de pruebas (`fixtures`), incluyendo la conexión a la BD y la población/limpieza de datos.
 - `test_db_setup.py`: Contiene pruebas que verifican la estructura básica y los permisos de los roles.
 - `test_db_integrity.py`: Contiene pruebas que verifican la integridad de los datos (restricciones, claves foráneas, cascadas).

² <https://github.com/Digiliencia/InfraestructuraDigiliencia>

El diagrama entidad-relación de la base de datos se muestra a continuación junto a la descripción de las tablas.



La tabla USERS almacena la información de los usuarios registrados.

Columna	Tipo	Descripción
ID	SERIAL	Clave Primaria. Identificador único.
email	VARCHAR(255)	Único, No Nulo. Correo electrónico del usuario.
password	VARCHAR(255)	No Nulo. Hash de la contraseña.

La tabla CHATS almacena conversaciones, asociados a un usuario.

Columna	Tipo	Descripción
ID	SERIAL	Clave Primaria. Identificador único del chat.
título	VARCHAR(255)	Título o nombre del chat.
user_id	INTEGER	No Nulo, Clave Foránea a USERS(ID). ON DELETE CASCADE.

La tabla IA_PROMPTS almacena plantillas de prompts del sistema para la IA.

Columna	Tipo	Descripción
ID	SERIAL	Clave Primaria.
prompt	TEXT	No Nulo. El texto del prompt del sistema.
prompt_description	TEXT	Descripción del propósito del prompt.
IA_name	VARCHAR(255)	Único, No Nulo. Nombre para el prompt.

La tabla Models almacena los diferentes modelos de IA disponibles.

Columna	Tipo	Descripción
ID	SERIAL	Clave Primaria.
IA_name	VARCHAR(255)	Único, No Nulo. Nombre del modelo (ej. 'GPT-4').

La tabla MESSAGES almacena cada mensaje individual dentro de un chat.

Columna	Tipo	Descripción
ID	SERIAL	Clave Primaria.
order_number	INTEGER	No Nulo. Orden del mensaje en el chat. Único con chat_id.
content	TEXT	No Nulo. El contenido del mensaje.
statistics	JSONB	Datos estadísticos para el mensaje (ej. tokens, tiempo).
chat_id	INTEGER	No Nulo, Clave Foránea a CHATS(ID). ON DELETE CASCADE.

model_id	INTEGER	Clave Foránea a MODELS(ID). ON DELETE RESTRICT.
ia_prompt_id	INTEGER	Clave Foránea a IA_PROMPTS(ID). ON DELETE RESTRICT.

También se han elaborado una vista y una función:

- **users_id_email_view**: vista segura sobre la tabla USERS que solo expone las columnas ID y email. Su propósito es permitir que la aplicación consulte información de usuario sin exponer datos sensibles como la columna de contraseña.
- **get_user_id_by_email(user_email VARCHAR)**: Función que recibe un email como parámetro y devuelve el ID de usuario correspondiente. Si el email no se encuentra en la base de datos, lanza una excepción (RAISE EXCEPTION).

Las variables de entorno, almacenadas en el fichero .env son las siguientes:

- Utilizadas por la imagen oficial de PostgreSQL.
- POSTGRES_PASSWORD: Contraseña para el superusuario postgres.
- POSTGRES_USER: Nombre del superusuario (el valor por defecto es postgres).

Variables Personalizadas

Utilizadas por los scripts de inicialización.

- APP_DB_NAME: El nombre de la base de datos de la aplicación (ej. diligencia).
- DB_OWNER_USER: El rol que es dueño de la base de datos y todos sus objetos.
- DB_OWNER_PASSWORD: Contraseña para el rol owner.
- APP_USER: El rol que la aplicación principal utilizará para sus operaciones (leer/escribir chats, etc.).
- APP_USER_PASSWORD: Contraseña para el rol APP_USER.
- APP_USER_LOGIN: Un rol específico para la gestión de cuentas (leer/escribir en la tabla USERS).
- APP_USER_LOGIN_PASSWORD: Contraseña para el rol APP_USER_LOGIN.

Aplicación web

La estructura de la aplicación está organizada en módulos para una clara separación de responsabilidades, siguiendo las prácticas recomendadas de FastAPI, biblioteca empleada para la implementación de los puntos de acceso. Este módulo aún está en desarrollo y sujeto a cambios.

La estructura de directorios de la aplicación web es la siguiente.

- **routers/**: Contiene los archivos que definen los **puntos de acceso (endpoints)** de la API. Cada archivo agrupa rutas relacionadas para una mejor organización.
 - chats.py: Define los endpoints relacionados con la gestión de **conversaciones y mensajes** (/chats, /conversations).

- custom_users.py: Contiene **endpoints personalizados** para la gestión de usuarios, como la eliminación de la cuenta o la exportación de datos, que no están cubiertos por la biblioteca de autenticación estándar.
- **auth/**: Módulo dedicado a la **autenticación y gestión de usuarios**.
 - db.py: Adaptador de base de datos para FastAPI-Users, la biblioteca utilizada para la autenticación. Se encarga de la interacción de la lógica de usuarios con la base de datos.
 - manager.py: Contiene la lógica del **gestor de usuarios**, controlando acciones como el registro, la verificación y el restablecimiento de contraseñas.
 - transport.py: Configura el método de transporte de tokens, en este caso **JSON Web Token (JWT)**, para el intercambio seguro de credenciales.
- **core/**
 - config.py: Archivo para cargar y validar las **variables de entorno** del proyecto utilizando **Pydantic**, lo que garantiza que la configuración sea robusta y segura.
- **db/**: Módulo de la **base de datos**.
 - models.py: Define los modelos **ORM (Mapeo Objeto-Relacional)** de **SQLAlchemy**. Estos modelos reflejan la estructura de las tablas de la base de datos (modelo Entidad-Relación) en objetos Python.
 - session.py: Configura la **sesión de base de datos asíncrona**, permitiendo una comunicación eficiente con la base de datos en un entorno asíncrono.
- **schemas/**: Contiene los **esquemas de datos** del proyecto definidos con **Pydantic**. Estos esquemas garantizan la validación y serialización de los datos de entrada y salida de la API.
 - chat.py: Esquemas para los datos relacionados con los **chats**.
 - user.py: Esquemas para los datos de los **usuarios**.
- **.env**: Un archivo con las **variables de entorno** de la aplicación.
- **main.py**: El **punto de entrada** de la aplicación. Aquí se inicializa la instancia de FastAPI, se registran los **middlewares** (intermedios) y se incluyen todos los routers de la API.
- **requirements.txt**: Enumera todas las **dependencias de Python** del proyecto, asegurando que el entorno de desarrollo y producción sea consistente y reproducible.

A continuación, se detallan los puntos de acceso (endpoints) de la API. Todos los puntos de acceso se encuentran bajo el prefijo /api.

Los *endpoints* de autenticación (/auth) gestionan la autenticación y la gestión de cuentas de usuario.

- POST /api/auth/login
 - **Autenticación**: No requerida.
 - **Descripción**: Permite a un usuario iniciar sesión usando su correo electrónico y contraseña. Si las credenciales son correctas, el sistema devuelve un **token de sesión** que debe ser utilizado en las futuras peticiones para acceder a los endpoints protegidos.
- POST /api/auth/logout
 - **Autenticación**: Requerida.

- **Descripción:** Cierra la sesión del usuario actual. Esto invalida el token de sesión, haciendo que el usuario tenga que volver a iniciar sesión para realizar acciones que requieran autenticación.
- POST /api/auth/register
 - **Autenticación:** No requerida.
 - **Descripción:** Crea una nueva cuenta de usuario. El sistema registra el correo electrónico y la contraseña proporcionados, y generalmente envía un correo de verificación para activar la cuenta.
- POST /api/auth/verify/{token}
 - **Autenticación:** No requerida.
 - **Descripción:** Confirma la dirección de correo electrónico de un usuario utilizando un token de verificación que se envía por correo electrónico durante el proceso de registro. Este paso es crucial para activar la cuenta.
- POST /api/auth/forgot-password
 - **Autenticación:** No requerida.
 - **Descripción:** Inicia el proceso de recuperación de contraseña. El usuario introduce su correo electrónico y el sistema envía un enlace o token para restablecer la contraseña a esa dirección.
- POST /api/auth/reset-password/{token}
 - **Autenticación:** No requerida.
 - **Descripción:** Permite al usuario establecer una nueva contraseña. El usuario debe proporcionar la nueva contraseña junto con un token de recuperación válido, recibido previamente por correo electrónico.

Los *endpoints* de usuarios (/users) se utilizan para gestionar el perfil y los datos del usuario autenticado.

- GET /api/users/me
 - **Autenticación:** Requerida.
 - **Descripción:** Devuelve los datos del perfil del usuario que ha iniciado sesión, como su correo electrónico, nombre de usuario y otros detalles relevantes.
- PATCH /api/users/me
 - **Autenticación:** Requerida.
 - **Descripción:** Permite al usuario autenticado actualizar su información de perfil. La solicitud PATCH indica que solo se actualizarán los campos proporcionados en el cuerpo de la petición.
- DELETE /api/users/me
 - **Autenticación:** Requerida.
 - **Descripción:** Elimina de forma permanente la cuenta del usuario autenticado y todos los datos asociados, como sus conversaciones. Esta acción es irreversible.
- GET /api/users/{id}/export
 - **Autenticación:** Requerida.
 - **Descripción:** Permite a un usuario exportar una copia de todos sus datos personales almacenados en el sistema, lo que cumple con el derecho a la portabilidad de datos. Este endpoint es una funcionalidad personalizada de la API.

Chats y Conversaciones

Los *endpoints* de chats y conversaciones gestionan la interacción con las conversaciones y los mensajes de la IA.

- GET /api/conversations
 - **Autenticación:** Requerida.
 - **Descripción:** Devuelve una lista de las conversaciones de chat del usuario actual. Generalmente, esta lista incluye metadatos como el ID y el título de cada conversación.
- GET /api/chats/{chat_id}
 - **Autenticación:** Requerida.
 - **Descripción:** Recupera el contenido completo de una conversación específica. El chat_id identifica de forma única la conversación.
- PATCH /api/chats/{chat_id}
 - **Autenticación:** Requerida.
 - **Descripción:** Envía un nuevo mensaje o pregunta a una conversación existente y recibe una respuesta de la IA. Esta operación actualiza el chat con el nuevo intercambio de mensajes.
- PUT /api/chats/{chat_id}
 - **Autenticación:** Requerida.
 - **Descripción:** Permite importar o reemplazar una conversación completa. Esta funcionalidad es útil para sincronizar o transferir chats, reemplazando todo el contenido existente con el proporcionado en el cuerpo de la petición.
- DELETE /api/chats/{chat_id}
 - **Autenticación:** Requerida.
 - **Descripción:** Elimina una conversación de chat específica, identificada por el chat_id, junto con todos los mensajes que contiene.

Lógica de negocio

Arquitectura y organización de la lógica de negocio

Este informe describe la arquitectura de software diseñada para el proyecto, estructurada en capas que diferencian el dominio, los servicios, la infraestructura, la aplicación y la interacción con modelos de lenguaje. La separación en capas permite mantener independencia entre componentes, mejorar la mantenibilidad y facilitar la evolución del sistema.

Modelos (Dominio y Persistencia)

Los modelos constituyen la base del dominio y son implementados con neomodel, lo que permite definir entidades directamente sobre Neo4j. Entre los archivos principales se encuentran:

- `news.py`: entidad central que representa noticias.
- `topic.py`: entidad que modela tópicos o temas.
- `field.py`: entidad que representa campos temáticos.
- Además de las definiciones estructurales, los modelos incorporan métodos utilitarios. Destaca `News.get_or_create_with_relationships`, encargado de:
 - Crear instancias de entidades en Neo4j.
 - Gestionar relaciones entre ellas.
 - Incluir opcionalmente embeddings para enriquecer los datos.

Servicios de Dominio

Los servicios encapsulan la lógica de negocio y orquestación de procesos. Se dividen en los siguientes módulos:

- **`news_service.py`**: coordina la creación de entidades a partir de procesos de scraping, asigna tópicos y campos, y gestiona tanto embeddings como fragmentos de texto (chunks).
- **`topic_service.py` y `field_service.py`**: responsables de la obtención y estructuración de datos, en forma de listas o jerarquías.
- **`topic_classification_service.py` y `field_classification_service.py`**: adaptan el motor base de clasificación para las entidades de tópicos y campos.
- **Motor de clasificación (`base_classification_service.py`)**: núcleo reutilizable del sistema de clasificación. El flujo estándar de este motor incluye:
 1. Generación de prompt.
 2. Llamada al modelo de lenguaje (LLM).
 3. Extracción de salida en formato JSON.
 4. Mapeo a objetos de dominio.
 5. Filtrado de resultados.

Prompts (Formulación de Instrucciones)

La formulación de instrucciones se encuentra desacoplada en módulos dedicados:

- `topic_classification_prompt.py`.
- `field_classification_prompt.py`.

Estos archivos encapsulan prompts diseñados para minimizar alucinaciones, asegurando resultados consistentes y alineados con las entidades del dominio.

Utilidades Transversales

Se implementan herramientas de soporte que garantizan consistencia en distintos procesos, entre las cuales destacan:

- `llm.py`: encargado de parseo robusto de respuestas generadas por LLM.

- `utils/time.py`: utilidades de temporización y gestión temporal.

Orquestación y Casos de Uso (Aplicación)

La capa de aplicación se implementa mediante scripts de línea de comandos (`cmd`) que permiten ejecutar flujos completos de procesamiento. Ejemplos destacados incluyen:

- `scrap.py`: realiza scraping, persiste los datos y desencadena el flujo de clasificación.
- `classify_unfielded_news.py`: clasifica noticias sin campo asignado y las enriquece con embeddings y fragmentos.

Estos scripts encadenan secuencias de procesos: scraping, persistencia, clasificación y enriquecimiento.

Embeddings y Enriquecimiento

La gestión de embeddings y fragmentación de texto se centraliza en `news_service`, que ofrece métodos de procesamiento en batch. El proceso de chunking se delega al `ChunkService`, invocado para dividir texto en fragmentos y permitir un análisis más granular.

Agente y RAG (Fase en desarrollo)

En el directorio `agents/tools/` se encuentran actualmente utilidades auxiliares. La hoja de ruta contempla su evolución hacia tools formales que permitan desplegar un flujo completo de Retrieval-Augmented Generation (RAG), con integración directa a agentes inteligentes.

Separación Conceptual

La arquitectura mantiene una separación conceptual clara:

- **Dominio:** modelos de negocio y persistencia.
- **Servicios:** reglas de negocio y orquestación.
- **Infraestructura:** integración con Neo4j y comunicación HTTP con modelos de lenguaje.
- **Aplicación:** scripts de línea de comandos para casos de uso concretos.
- **Presentación al LLM:** prompts especializados para tareas de clasificación y enriquecimiento.

Gestión del Proyecto

Esta sección contiene los documentos e informes relativos a la definición y gestión del proyecto además de documentos varios.

Actas de reunión

Actas de las reuniones realizadas. No se empezaron a realizar hasta el mes de marzo.

21 de marzo de 2025

Hora de Inicio: 10:00
Hora de Fin: 12:25
Ubicación: Híbrida (Seminario y Online)

Miembros convocados: Enrique, Esteban, Carlos, Álvaro, Ander, César (online), Santiago (online) y Gabriel (online).

Se confirma la asistencia de todos los participantes.

Orden del Día

1. Transparencia y documentación del proyecto.
2. Herramientas y metodología para la gestión del proyecto.
3. Uso de herramientas de IA (ChatGPT).
4. Planificación del escapeo y mapeo de carreras de ciberseguridad.

Resumen de las Deliberaciones

La reunión comienza tratando la necesidad de mejorar la **transparencia y la gestión** del proyecto. Se discute la importancia de documentar todo el trabajo realizado para mantener un registro claro del progreso. Se evalúan diferentes herramientas de gestión, como Taskade y Github, para centralizar las tareas y la documentación.

Finalmente, se definen las próximas tareas técnicas. Se discute el uso de ChatGPT y se planifica el proceso de escapeo de información, incluyendo el Esquema Nacional de Seguridad (ENS) y carreras de ciberseguridad del NIST.

Acuerdos y Conclusiones

Tras la puesta en común, se establecen los siguientes puntos y se asignan las siguientes tareas:

- **Gestión y Documentación:**
 - Se acuerda mejorar la transparencia del proyecto, documentando el trabajo.
 - Se elige **Github** como la herramienta oficial para la gestión de tareas.
 - Ander asume el rol de secretario para las próximas reuniones.
- **Tareas Técnicas Asignadas:**
 - **Gabriel:** Escapeo del Esquema Nacional de Seguridad (ENS) del CNI y mapeo de carreras de ciberseguridad.
 - **Álvaro:** Estudio de la herramienta MCP.
 - Se establece la necesidad de escapear también las imágenes asociadas a la información recopilada.

Sin más asuntos que tratar, se levanta la sesión a las 12:25 horas.

27 de marzo de 2025

Hora de Inicio: 10:00
Hora de Fin: 12:00
Ubicación: Seminario

Miembros convocados: Esteban, Carlos, Álvaro, Ander, César, Santiago y Gabriel.
Se confirma la asistencia única de Carlos y Ander.

Orden del Día

1. Gestión de tareas del proyecto.
2. Debate y fijación de los requisitos del proyecto.
3. Debate y fijación de la arquitectura inicial.
4. Establecimiento y planificación del primer *sprint*.
5. Propuesta para limitar la duración de las reuniones de seguimiento de los viernes.
6. Asignación de tareas relativas al primer *sprint*.

Resumen de las Deliberaciones

Reunión convocada para establecer las bases técnicas y de planificación del proyecto. Se inicia revisando la gestión de tareas, lo que lleva a la definición y corrección de la documentación de requisitos.

Una vez fijada la primera versión de los requisitos, se debate la arquitectura inicial del sistema. Con estos dos pilares definidos, se establece la duración y el alcance del primer *sprint* de desarrollo, asignando las tareas iniciales. Finalmente, se acuerda proponer una optimización en la duración de las reuniones generales de seguimiento.

Acuerdos y Conclusiones

- **Documento de Requisitos:** Se aprueba la primera versión del documento de requisitos tras corregir erratas y mover los puntos relativos a la interfaz de usuario a un documento anexo.
- **Documento de Arquitectura:** Se elaborará un documento de diseño con la arquitectura inicial decidida, que será presentado para su confirmación en la próxima reunión.
- **Primer Sprint:** Se establece la duración del primer *sprint* de desarrollo desde la fecha actual hasta finales de abril. Las tareas y responsables provisionales se recogen en el documento anexo tareas.pdf, pendientes de debate en la reunión del día 28.
- **Reuniones de Seguimiento:** Se acuerda proponer a Enrique la limitación temporal de la duración de las reuniones de los viernes.

Observaciones

- Se deja constancia de la ausencia justificada de Esteban y Gabriel.
- El resto de las ausencias no han sido notificadas al momento de la elaboración de la presente acta.

Sin más asuntos que tratar, se levanta la sesión a las 12:00 horas.

28 de marzo de 2025

Hora de Inicio: 10:00
Hora de Fin: 11:30
Ubicación: Seminario

Miembros convocados: Enrique, Esteban, Carlos, Álvaro, Ander, César, Santiago y Gabriel.

Se confirma la asistencia de todos los convocados excepto Esteban, cuya ausencia fue notificada. Se anota que Enrique se ausenta a las 11:15.

Orden del Día

1. Revisión y debate sobre la arquitectura del proyecto.
2. Discusión sobre la tecnología MCP.
3. Presentación de los avances del proyecto.
4. Definición del primer *sprint*.
5. Asignación de tareas del primer *sprint*.

Resumen de las Deliberaciones

La reunión da comienzo con la presentación de los avances generales del proyecto. A continuación, se abre un debate técnico sobre la arquitectura propuesta para el sistema y se analiza el estado de la investigación sobre la tecnología MCP.

Una vez revisados los puntos técnicos, el equipo procede a definir formalmente el alcance y los objetivos del primer *sprint* de desarrollo. Con el *sprint* definido, se realiza la asignación de las tareas correspondientes a cada miembro del equipo.

Acuerdos y Conclusiones

- **Tecnología MCP:** Álvaro continuará con la investigación de la viabilidad y aplicación de MCP en el proyecto.
- **Primer Sprint:** Quedan definidas y asignadas las tareas correspondientes al primer *sprint*.
- **Documentación:** Ander se encargará de crear dos documentos:
 - Un documento que defina formalmente el alcance y las tareas del primer *sprint*.
 - Un documento que describa la arquitectura general del proyecto acordada.

Sin más asuntos que tratar, se levanta la sesión a las 11:30 horas.

10 de abril de 2025

Hora de Inicio: 11:00

Hora de Fin: 12:30

Ubicación: Seminario

Miembros convocados: Enrique, Esteban, Carlos, Álvaro, Ander, César, Santiago, Gabriel y Sergio.

Se confirma la asistencia de todos los convocados. Se anota que Esteban se ausenta a las 11:45.

Orden del Día

1. Acceso a la plataforma SCAYLE.
2. Discusión sobre el protocolo A2A (Agent to Agent).
3. Revisión de la situación actual del proyecto (avances y bloqueos).
4. Análisis sobre la generación de vídeo mediante IA.
5. Estado del proceso de scrapeo mediante MCP.
6. Incorporación de Sergio al equipo de desarrollo.

Resumen de las Deliberaciones

La sesión comienza con una revisión del estado general del proyecto, donde el equipo expone los avances y los posibles bloqueos actuales.

Se confirma la incorporación de Sergio al equipo de desarrollo. En relación con este punto, se discute el acceso a la infraestructura de SCAYLE y se decide asignarle a Sergio el despliegue de la aplicación en sus servidores.

En el plano técnico, se aborda el desarrollo del protocolo A2A (Agent to Agent). Se informa de que el proceso de scrapeo de datos mediante MCP presenta dificultades que deben ser analizadas. Finalmente, se plantea como punto de mejora la necesidad de humanizar los textos generados por la inteligencia artificial.

Acuerdos y Conclusiones

Tras la puesta en común, se definen las siguientes acciones y puntos:

- Se asigna a Sergio del despliegue del proyecto en la plataforma de SCAYLE.
- Santiago se encargará de investigar y proponer métodos para humanizar el texto generado por la IA.
- Se toma nota de las dificultades en el scrapeo mediante MCP para reevaluar la estrategia actual.

Sin más asuntos que tratar, se levanta la sesión a las 12:30 horas.

2 de mayo de 2025

Hora de Inicio: 10:30

Hora de Fin: 12:30

Ubicación: Seminario

Miembros convocados: Enrique, Esteban, Carlos, Ander, César, Santiago, Gabriel, Manuel Domínguez (convocado a las 11:30), Jose Adserias (convocado a las 11:30) y Luca Pacioli (convocado a las 11:30).

Se confirma la asistencia de todos los miembros convocados. Se destaca que Manuel Domínguez, Jose Adserias y Luca Pacioli se incorporaron a la sesión a las 11:30 para tratar el punto específico sobre la colaboración con la Universidad de Salamanca, finalizando su participación a las 11:45.

Orden del Día

1. **Estado General del Proyecto:** Revisión de avances, hitos y posibles bloqueos.
2. **Evaluación de Modelos de IA:** Análisis y decisión sobre la migración de OpenAI (ChatGPT) a Google (Gemini).
3. **Desarrollo Web y Colaboración USAL:** Presentación de avances y objetivos de la colaboración con la Universidad de Salamanca.
4. **Planificación de Próximos Pasos y Asuntos Varios.**

Resumen de las Deliberaciones

La sesión, comienza con una revisión del estado actual del proyecto. Se confirman los avances según el cronograma previsto, sin identificar bloqueos críticos que impidan el desarrollo.

A continuación, se procede al análisis técnico sobre el modelo de Inteligencia Artificial empleado como asistencia en el desarrollo del proyecto. Se realiza una comparativa entre el modelo actual (ChatGPT de OpenAI) y el modelo Gemini de Google. Dado el acuerdo entre la universidad de León y Google acerca del uso de la IA Gemini, se opta por el uso de Gemini como modelo.

A las 11:30, se incorporan los Sres. Domínguez, Adserias y Pacioli para el tercer punto del día. Se presentan los requisitos del *front-end* y la lógica de negocio subyacente.

Finalmente, se tratan asuntos varios, donde se comunica la situación de un miembro del equipo (ver "Comentarios y Observaciones").

Acuerdos y Conclusiones

Tras las deliberaciones, se alcanzan los siguientes acuerdos:

1. **Migración a Google Gemini:** Se aprueba la sustitución del modelo de IA de OpenAI por el modelo Gemini de Google.
2. **Retraso de la colaboración con la USAL:** Se posterga el desarrollo del front-end a la espera de más avances y una definición mayor de los requisitos.
3. **Planificación de Demostración Interna:** Se programa una sesión de demostración interna para presentar los últimos avances, incluyendo la primera integración con Gemini. La fecha y hora fijadas son:

Jueves, 8 de mayo de 2025, de 12:00 a 14:00 horas.

6. Comentarios y Observaciones

- Se informa al equipo que Álvaro se ausentará del proyecto de forma temporal por motivos personales.

Sin más asuntos que tratar, se levanta la sesión a las 12:30 horas.

8 de mayo de 2025

Hora de Inicio: 12:00

Hora de Fin: 13:45

Ubicación: Reunión Virtual (vía Google Meet)

Miembros convocados: Enrique, Esteban, Carlos, Ander, César, Santiago, Gabriel, Manuel Domínguez, Jose Adserias y Carlos Estévez.

Se confirma la asistencia de todos los miembros convocados.

Orden del Día

1. Valoración de la necesidad de implementar un sistema de Gestión de Relaciones con Clientes (CRM) en el proyecto.
2. Análisis de implicaciones de seguridad y propuesta de consulta al Instituto Nacional de Ciberseguridad (INCIBE).
3. Evaluación preliminar de las plataformas CRM candidatas: Crisp.chat, HubSpot y Whistleblower.
4. Discusión sobre la viabilidad técnica, APIs y herramientas conectoras.
5. Acuerdos, asignación de tareas y planificación de la próxima sesión.

Resumen de las Deliberaciones

La reunión se centra en la posible necesidad de integrar una herramienta CRM sugerido por el equipo de la USAL.

Se inicia un debate sobre los requisitos funcionales y técnicos que debería cumplir la plataforma. Se propone realizar una consulta con el INCIBE, en especial con el equipo del servicio 017. A continuación, el equipo técnico presenta un análisis inicial de tres herramientas preseleccionadas: Crisp.chat, HubSpot y Whistleblower. Se discute también la necesidad real de incorporar estas tecnologías al proyecto.

Acuerdos y Conclusiones

Tras la puesta en común, se alcanzan los siguientes acuerdos:

1. **Rechazo del uso de CRM:** Finalmente se descarta el uso de estos sistemas en la fase actual del proyecto.

Sin más asuntos que tratar, se levanta la sesión a las 13:45 horas.

17 de junio de 2025

Hora de Inicio: 10:50
Hora de Fin: 12:00
Ubicación: Híbrida (Seminario y Online)

Miembros convocados: Enrique, Esteban, Carlos, Álvaro, Ander, César, Santiago, Gabriel y Sergio.

Asisten Enrique, Esteban, Carlos, César y Santiago de forma presencial, y Ander de forma online. Se registra la ausencia de Álvaro, Gabriel y Sergio.

Orden del Día

1. Definición de plazos para una prueba de concepto (PoC).
2. Discusión de funcionalidades de la plataforma:
 - a. Orientación profesional en ciberseguridad.
 - b. Matriz para evaluar el nivel de madurez de las empresas.
 - c. Catálogo de formación y competencias (España y Europa).
 - d. Análisis del impacto de tecnologías emergentes (cuántica, IA, metaverso).
 - e. Funciones adicionales: generación de vídeos, juegos didácticos.
 - f. Análisis de enfoques sectoriales (agrícola, universitario).

Resumen de las Deliberaciones

La reunión se centra en definir el alcance y los plazos para desarrollar una primera prueba de concepto funcional. Se realiza una sesión de brainstorming para evaluar las distintas funcionalidades que podría incluir la plataforma, abarcando desde la orientación profesional y la creación de un catálogo formativo, hasta herramientas más complejas como una matriz de madurez para empresas.

Se debate también el impacto de tecnologías emergentes en el sector y la posibilidad de incorporar funciones innovadoras como la generación de vídeos personalizados. Finalmente, se discuten los aspectos técnicos para el despliegue de la PoC, incluyendo la infraestructura de alojamiento temporal para el LLM.

Acuerdos y Conclusiones

- **Prueba de Concepto (PoC):** Se acuerda desarrollar una primera prueba de concepto en un plazo de dos meses, debiendo estar finalizada antes de septiembre.
- **Infraestructura Temporal:** Se utilizará la infraestructura de Oracle para alojar el LLM del proyecto hasta que se formalice el acuerdo definitivo con SCAYLE.
- **Acción Inmediata:** Se procederá con la migración del proyecto a la plataforma de Oracle para comenzar el desarrollo de la PoC.

Sin más asuntos que tratar, se levanta la sesión a las 12:00 horas.

Otros documentos

Especificación de Requisitos

El presente documento pretende fijar y dejar constancia de los objetivos y el alcance del proyecto. A continuación, se recoge la especificación general y técnica del proyecto, donde se detalla el resultado a obtener.

Descripción General

Se desarrollará un servicio web basado en inteligencia artificial conversacional (LLM) para atender consultas relacionadas con ciberseguridad. Este servicio proporcionará respuestas, información, consejos y guías sobre incidentes, medidas de protección, amenazas y otros temas tecnológicos. Adicionalmente, ofrecerá orientación de calidad sobre el desarrollo profesional en el ámbito de la ciberseguridad.

Una característica clave del servicio será la adaptación al usuario, personalizando las respuestas según su nivel de conocimiento, situación social o discapacidad. El perfil se construirá a partir de la información que el usuario proporcione explícitamente y del análisis de la propia conversación. Esta adaptación se reflejará en el registro del lenguaje, el uso de recursos audiovisuales, la complejidad de la información y la granularidad de las instrucciones, con el objetivo de facilitar la comprensión y maximizar la utilidad.

El servicio tomará en cuenta el contexto de la conversación para generar respuestas coherentes y relevantes, basándose siempre en la situación actual en el momento de la consulta. Para ello, se mantendrá una base de datos actualizada de forma continua y automatizada (mediante técnicas de *scraping*) con noticias, informes y artículos de fuentes fiables. Las respuestas incluirán citas a dichas fuentes para garantizar su fiabilidad.

El sistema implementará filtros para no responder a consultas fuera del ámbito de la ciberseguridad o que sean de naturaleza impertinente, inmoral o ilegal. En casos que requieran atención especializada, remitirá al usuario a las autoridades competentes (como el **servicio 017** de INCIBE), pudiendo incluso transferir la conversación a un operador humano.

Otras consideraciones importantes son:

- El servicio estará alojado en la infraestructura propia de la organización sobre un sistema operativo Linux.
- La arquitectura será escalable para atender a un gran número de usuarios simultáneos.
- La interfaz web cumplirá con los estándares de accesibilidad y diseño adaptativo para dispositivos móviles.
- El servicio estará disponible en inglés, castellano y las lenguas cooficiales del Estado.
- Se generarán datos estadísticos para auditorías y se elaborará una guía de usuario.

Especificación de Requisitos

A continuación, se detalla la especificación técnica de los requisitos del sistema.

Requisitos Funcionales

Definen la funcionalidad específica que el sistema debe realizar.

1. **Atención a Consultas de Ciberseguridad:** El servicio resolverá dudas y proporcionará información, consejos y guías sobre ciberseguridad.
2. **Adaptación al Usuario:** El sistema personalizará las respuestas basándose en el perfil del usuario, construido a partir de datos explícitos y del contexto de la conversación.
3. **Generación de Respuestas Contextualizadas:** Las respuestas considerarán el historial de la conversación y la información más reciente disponible.
4. **Base de Datos Dinámica:** El contenido de la base de conocimiento se actualizará de forma periódica y automática mediante *scraping* de fuentes de internet.
5. **Configuración de Parámetros por el Usuario:** El usuario podrá ajustar características de la respuesta, como la "temperatura" (creatividad) o el formato (texto, gráficos).
6. **IA Especializada:** El modelo contará con especializaciones para temas concretos, como la **orientación profesional en ciberseguridad**, dotando a la IA de herramientas y contexto para ser más eficiente.
7. **Acceso sin Identificación:** El servicio principal podrá ser utilizado sin necesidad de crear una cuenta de usuario.
8. **Almacenamiento de Conversaciones:** El servicio permitirá a los usuarios registrados guardar sus conversaciones para retomarlas posteriormente.
9. **Eliminación de Conversaciones:** Los usuarios registrados podrán eliminar su historial de conversaciones.
10. **Orientación Profesional:** El servicio ofrecerá guías de calidad sobre el desarrollo de una carrera profesional en ciberseguridad.
11. **Justificación de Respuestas:** Las respuestas generadas por la IA citarán las fuentes fiables que respaldan la información.
12. **Remisión a Asesoramiento Especializado:** En situaciones que lo requieran, el servicio remitirá al usuario a instituciones competentes o transferirá la conversación a un operador humano (servicio 017).
13. **Asistencia al Servicio 017:** El servicio podrá ser utilizado como herramienta de apoyo por los operadores del 017.
14. **Generación de Datos Estadísticos:** El sistema generará registros y estadísticas de uso para auditoría y mejora continua.
15. **Guía de Usuario:** Se elaborará una guía de usuario detallada y adaptada al público general.

Requisitos No Funcionales

Establecen los criterios de calidad y el funcionamiento general del sistema.

Plataforma y Arquitectura

1. **Arquitectura Web:** El sistema será un servicio web alojado en la infraestructura de la organización y accesible desde un navegador.
2. **Compatibilidad:** El servicio será compatible con las últimas versiones de los principales navegadores web (Chrome, Firefox, Safari, Edge).
3. **Facilidad de Mantenimiento:** La arquitectura modular del sistema deberá facilitar las actualizaciones y el mantenimiento.

Experiencia de Usuario

4. **Tiempo de Respuesta:** El sistema ofrecerá tiempos de respuesta rápidos para garantizar una interacción fluida.
5. **Usabilidad:** El diseño de la interfaz y la interacción con el servicio serán intuitivos y fáciles de usar.
6. **Accesibilidad:** La interfaz web cumplirá con los estándares de accesibilidad vigentes (WCAG).
7. **Idiomas:** El servicio y la documentación estarán disponibles en inglés, castellano y las lenguas cooficiales del Estado.

Rendimiento y Escalabilidad

8. **Escalabilidad:** La arquitectura debe ser capaz de escalar horizontalmente para gestionar un aumento en el número de usuarios simultáneos.
9. **Calidad del Servicio bajo Carga:** En picos de demanda, el servicio podrá restringir funcionalidades no esenciales para garantizar la disponibilidad de las funciones básicas.
10. **Fiabilidad y Disponibilidad:** El servicio deberá garantizar un alto tiempo de disponibilidad (uptime).
11. **Actualización sin Interrupción:** La actualización de la base de datos de conocimiento se realizará sin afectar la disponibilidad del servicio.

Requisitos Legales y de Cumplimiento

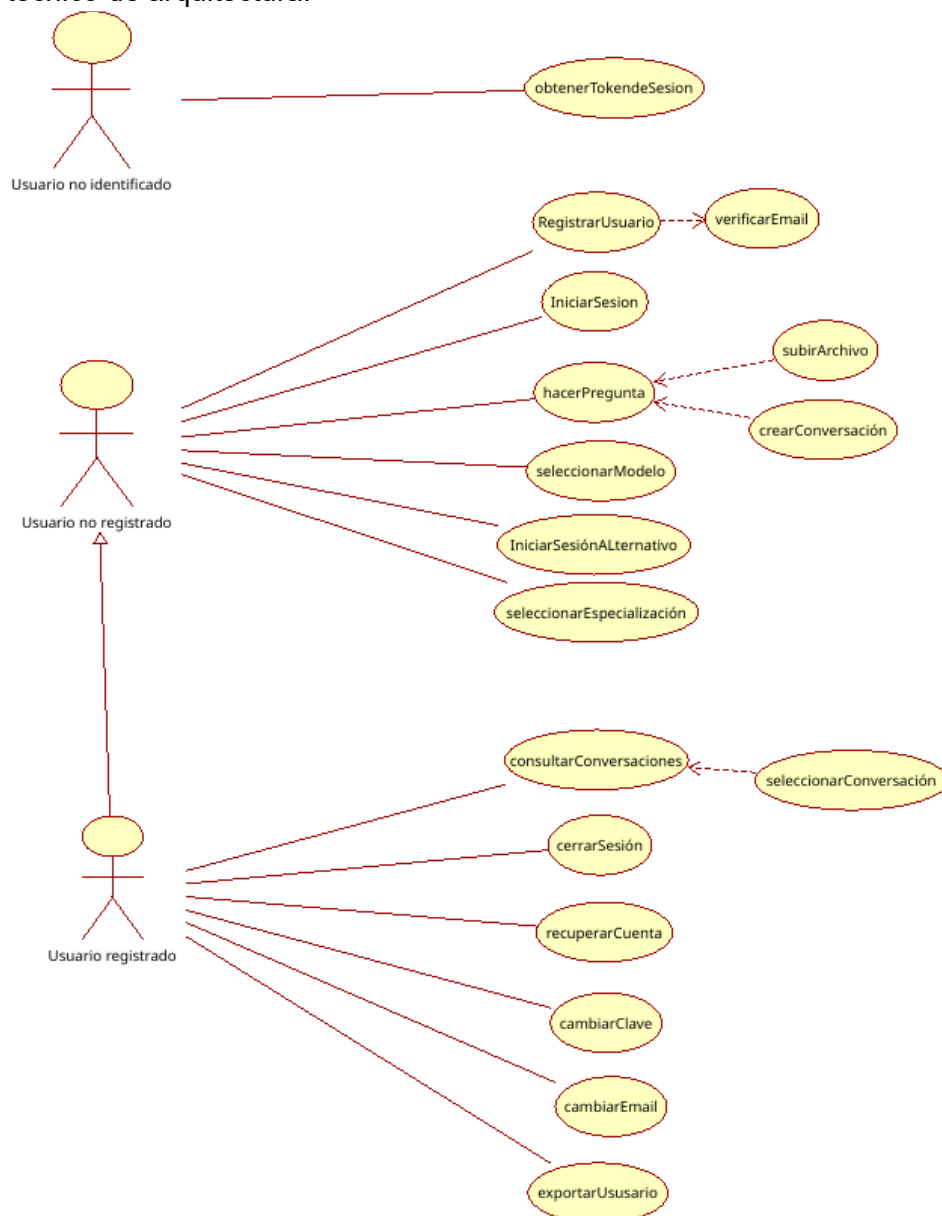
1. **Privacidad y Protección de Datos:** El servicio cumplirá con la normativa vigente sobre protección de datos (RGPD).
2. **Transparencia de Datos del Usuario:** Los usuarios registrados podrán solicitar y recibir toda la información que el sistema almacena sobre ellos.
3. **Cumplimiento Normativo General:** El servicio se adherirá a todas las regulaciones aplicables en materia de accesibilidad, privacidad y otros aspectos legales.

Requisitos de Seguridad

1. **Seguridad Integral:** El servicio implementará medidas para proteger la disponibilidad, confidencialidad e integridad de la plataforma y los datos.
2. **Protección contra Abusos y Ataques:** El sistema deberá ser resistente a ataques comunes (inyección, XSS, etc.) y a intentos de abuso.
3. **Restricción de Contenido:** El servicio implementará filtros para evitar responder a consultas fuera de su ámbito, así como a preguntas impertinentes, inmorales o ilegales.

Casos de Uso (CdU)

La funcionalidad del servicio desde el punto de vista del usuario se ilustra en el siguiente diagrama de casos de uso. Una descripción detallada de los mismos se encuentra en el documento técnico de arquitectura.



Arquitectura del Sistema

Estructura del Proyecto

Este documento presenta la estructura general del sistema, así como su justificación. No obstante, está sujeto a cambios a medida que el proyecto evolucione.

Arquitectura

La estructura del proyecto seguirá un **esquema en tres capas** (presentación, lógica de negocio y persistencia) y estará compuesta por módulos aislados en contenedores (**Docker**) bajo un esquema cliente-servidor. Todos los contenedores residirán en un servidor propio.

Los componentes principales del sistema serán:

- Módulos de bases de datos (SQL y orientada a grafos).
- Módulo del modelo LLM.
- Módulo de la lógica de negocio (agente).
- Módulo del servidor web.
- Módulo de scrapeo.
- Módulos auxiliares del agente.

El lado del cliente consistirá en una interfaz web que gestionará la interacción del usuario con el servicio. Se definirá una **API** para estandarizar la comunicación entre los diferentes módulos.

Base de Datos

Se utilizarán dos bases de datos para cometidos diferentes, optimizando el almacenamiento y la consulta de información:

- **Base de Datos No Relacional (Orientada a Grafos):** Almacenará el conocimiento extraído del proceso de escrapeo. Su estructura en grafo es ideal para ser consumida por el modelo de lenguaje, permitiendo una generación de respuestas más rica y contextual.
- **Base de Datos Relacional (SQL):** Almacenará datos estructurados como estadísticas de uso, información de usuarios, credenciales y registros de conversaciones.

Servicio Web

La aplicación web se encargará de la gestión de usuarios, sesiones y la presentación de la información. La generación de respuestas y la lógica compleja serán delegadas al módulo de lógica de negocio.

La comunicación entre el cliente y el servidor se realizará principalmente mediante una API REST. Para mejorar la experiencia de usuario durante la conversación, se emplearán Web Sockets para la recepción de los mensajes del LLM, permitiendo un *streaming* continuo de la respuesta en lugar de esperar a que esta se genere por completo.

Módulo LLM

Este módulo contendrá únicamente el modelo de lenguaje y sus recursos esenciales. Se definirá una API para su interacción exclusiva con el módulo de lógica de negocio. La arquitectura soportará el uso de múltiples modelos de lenguaje de forma simultánea.

Lógica de Negocio (Agente)

Este módulo es el núcleo del sistema. Se encarga de orquestar la interacción entre el servidor web, el LLM y los módulos auxiliares. Representa toda la lógica de la aplicación, a excepción del propio modelo de lenguaje.

Módulo de Scrapeo

Este componente se encargará de escraper multitud de páginas web y recursos en línea. Incluirá la lógica necesaria para ejecutar el proceso de forma periódica, manteniendo la base de datos de conocimiento (el grafo) constantemente actualizada.

Módulos Auxiliares

Estos módulos representan una división más conceptual que práctica y estarán a disposición del agente. Su naturaleza puede ser muy variada, desde modelos generadores de imágenes o vídeos hasta simples *prompts*. Dependiendo de su envergadura, podrán implementarse en contenedores independientes o estar incluidos junto al módulo de lógica de negocio.

Especializaciones

Las configuraciones especializadas se basarán principalmente en *prompts* predefinidos y herramientas concretas que podrán ser específicas de la especialización o modificaciones de las generales.

Justificación de la Arquitectura

- **Cliente-Servidor y 3 Capas:** Es el estándar de facto para servicios web, permitiendo una clara separación entre la presentación (*frontend*) y la lógica y persistencia (*backend*), lo que facilita el desarrollo y mantenimiento.
- **Contenerización (Docker):** El uso de contenedores para los diferentes módulos (microservicios) garantiza la portabilidad, facilita la gestión de dependencias y mejora la seguridad del sistema gracias al aislamiento de los servicios.
- **API REST:** Como interfaz de comunicación principal, es un protocolo simple, eficiente y ampliamente utilizado, lo que facilita su implementación, mantenimiento y protección. El uso de APIs internas entre módulos limita el acoplamiento y reduce la superficie de ataque.
- **Dualidad de Bases de Datos:** Las BD orientadas a grafos son ideales para representar conocimiento complejo y ser usadas por modelos de IA, mientras que las BD SQL son óptimas para datos transaccionales y estructurados.
- **Segregación de Módulos Auxiliares:** La estructura flexible de las herramientas del agente permite adaptar la arquitectura a la complejidad de cada herramienta, integrándolas en el módulo de lógica de negocio si son simples, o desplegándolas en contenedores propios si son servicios complejos.

Casos de Uso (CdU)

A continuación, se presentan los casos de uso de la aplicación desde la perspectiva del usuario.

Actores	
Nombre	Descripción
Usuario no autenticado	Usuario que no posee un token de sesión válido.
Usuario no registrado	Usuario que no ha iniciado sesión, pero posee un token de sesión válido.
Usuario registrado	Usuario que ha iniciado sesión en el sistema y cuenta con un token válido.

Acciones			
Nombre	Descripción	Precondición	Postcondición
Obtener Token	El sistema crea y envía un token de sesión. El usuario pasa a ser no registrado.	El usuario no posee un token válido.	El usuario recibe un token de sesión.
Registrar Usuario	El usuario se registra con email y contraseña, verifica su correo y obtiene acceso.	El usuario no ha iniciado sesión.	El usuario queda registrado e inicia sesión.
Iniciar Sesión	El usuario introduce sus credenciales y un CAPTCHA para obtener un token de sesión.	El usuario no está autenticado.	El usuario queda autenticado.
Inicio Sesión Alternativo	El usuario se autentica a través de una red social (Google, etc.).	El usuario no está autenticado.	El usuario queda autenticado.
Hacer Pregunta	El usuario envía texto al modelo y recibe una respuesta.	El usuario está autenticado.	El usuario recibe una respuesta.
Recuperar Cuenta	El usuario solicita un enlace para restablecer su contraseña a través de su email.	El usuario no está autenticado.	El usuario recibe un enlace o código.
Cerrar Sesión	El sistema invalida el token de sesión actual del usuario.	El usuario está autenticado.	El usuario deja de estar autenticado.
Cambiar Clave	El usuario cambia su contraseña, lo que invalida la sesión actual.	El usuario está autenticado.	La contraseña es modificada.
Cambiar Email	El usuario cambia su email asociado, lo que invalida la sesión actual.	El usuario está autenticado.	El correo electrónico es modificado.
Consultar Conversaciones	El sistema devuelve una lista con los títulos de las conversaciones del usuario.	El usuario está autenticado.	El usuario recibe los títulos.

Verificar Email	El usuario usa el enlace de verificación para activar su cuenta.	Ha recibido un correo de verificación.	La cuenta queda activada.
Seleccionar Conversación	El usuario carga una conversación anterior para continuar la interacción.	El usuario está autenticado.	El usuario recibe la conversación.
Seleccionar Modelo	El usuario elige con qué modelo de IA interactuar.	El usuario está autenticado.	Se establece un nuevo modelo.
Seleccionar Especialización	El usuario elige una especialización, lo que crea una nueva conversación con un prompt específico.	El usuario está autenticado.	Se establece una especialización.
Crear Conversación	El sistema crea una nueva conversación. Si no está autenticado, la anterior se elimina.	El usuario está autenticado.	Se crea una nueva conversación.

Especificación de Requisitos de WikIA

Introducción

El proyecto WikIA tiene como objetivo establecer una enciclopedia de ciberseguridad, análoga a Wikipedia, utilizando el software MediaWiki como plataforma base. Esta enciclopedia será principalmente generada y mantenida por un Modelo de Lenguaje Grande (LLM), cumpliendo una doble función: servir como base de conocimiento interna para las consultas del propio modelo y como una fuente de datos estructurada y accesible al público sobre conceptos de ciberseguridad.

Requisitos Generales del Sistema

A continuación, se detallan los requisitos que definen el alcance y la funcionalidad del sistema WikIA.

Requisitos Funcionales

Estos requisitos describen las capacidades y acciones que el sistema debe poder realizar.

1. **Generación y Edición Autónoma de Contenido:** El LLM deberá ser capaz de crear nuevas entradas (artículos) y modificar las existentes de forma autónoma, basándose en información adquirida de diversas fuentes, como el proceso de *scrapeo* web.
2. **Edición y Supervisión por Desarrolladores:** El personal desarrollador deberá disponer de las herramientas necesarias para visualizar, editar y verificar toda la información de la enciclopedia. Las modificaciones podrán realizarse de dos formas:
 - a. **General:** Instruyendo al modelo, mediante lenguaje natural o de forma explícita, para que actualice la enciclopedia según su criterio.
 - b. **Específica:** Modificando directamente entradas individuales o conjuntos de ellas.
3. **Flujo de Verificación y Publicación:** Las adiciones o modificaciones realizadas por el LLM no serán publicadas ni utilizadas por el modelo principal de consulta de forma inmediata. Deberán generar una nueva versión o borrador que un desarrollador deberá verificar y aprobar explícitamente. Este mecanismo es crucial para prevenir el "envenenamiento" de la base de conocimiento.
4. **Sistema de Control de Versiones:** La plataforma deberá implementar un sistema de control de versiones que permita observar, comparar y auditar todos los cambios realizados en el contenido, tanto por el LLM como por los desarrolladores.
5. **API de Acceso Programático:** Se deberá implementar una API para permitir el acceso del LLM a la Wiki. Esta API podría tener funcionalidades distintas a la interfaz de usuario y facilitará la integración futura con otras herramientas.
6. **Generación Programada de Contenido:** El sistema deberá permitir la configuración de rutinas para que el LLM genere o actualice contenido en intervalos de tiempo definidos (ej. diariamente, semanalmente).

Requisitos de Contenido y Datos

Estos requisitos definen las características y reglas de la información almacenada.

1. **Alcance Temático:** La enciclopedia se centrará exclusivamente en información relacionada con la ciberseguridad en un sentido amplio. Esto incluye datos técnicos específicos del campo, así como información contextual necesaria para su comprensión (ej. conceptos de informática básica, material de divulgación y formación).
2. **Referenciación y Citas de Fuentes:** Toda la información generada y almacenada en la enciclopedia deberá estar correctamente referenciada y respaldada por fuentes de información fiables.
3. **Niveles de Certeza ("Grados de Verdad"):** La información podrá ser catalogada con diferentes niveles de certeza. Este atributo determinará la propensión del LLM a modificar una entrada. Las modificaciones realizadas manualmente por un desarrollador recibirán el nivel de certeza más alto, pudiendo ser marcadas como inalterables por el modelo, con el objetivo de reducir la carga de trabajo en la revisión de cambios.

Requisitos No Funcionales

Estos requisitos definen los atributos de calidad del sistema.

1. **Rendimiento de Búsqueda y Acceso:** El acceso y la recuperación de información de la enciclopedia deberán ser rápidos y eficientes, tanto para las consultas del LLM como para el acceso de los usuarios finales, manteniendo el rendimiento a medida que el volumen de contenido crezca.
2. **Auditoría y Trazabilidad:** Se deberán registrar registros (*logs*) detallados de todas las modificaciones, verificaciones y aprobaciones de contenido, complementando el sistema de control de versiones.

Requisitos a Largo Plazo

Las siguientes funcionalidades se consideran deseables para futuras iteraciones del proyecto, pero no son parte del alcance inicial.

1. **Soporte Multilingüe:** La capacidad de la enciclopedia para generar y ofrecer contenido en múltiples idiomas.
2. **Contenido Interactivo y Multimedia:** La inclusión de elementos generados por IA, como vídeos explicativos cortos, simulaciones interactivas o laboratorios virtuales básicos para mejorar la comprensión de los usuarios.
3. **Personalización de la Experiencia:** Adaptación de la presentación de la información al perfil del usuario (nivel de conocimiento, edad, etc.), manteniendo siempre una versión estándar como base.

Especificación de la API REST

1. Introducción

Este documento define la interfaz de programación de aplicaciones (API) RESTful para la comunicación entre el cliente web y el servidor de lógica de negocio.

Convenciones Generales:

- **Formato de Datos:** Todas las peticiones y respuestas que contengan un cuerpo de datos utilizarán el formato application/json.
- **Autenticación:** Los endpoints que requieren autenticación deben recibir un Bearer Token en la cabecera HTTP Authorization.
 - Authorization: Bearer <token_de_sesion>
- **Parámetros de Ruta:** Los parámetros dinámicos en las rutas se indican con llaves, ej: /api/users/{userId}.

Endpoints de Autenticación y Usuarios

Esta sección describe los endpoints relacionados con la gestión de sesiones y cuentas de usuario.

Crear Sesión Anónima

- **Método:** POST
- **Ruta:** /api/auth/session
- **Acción:** Genera un token de sesión para usuarios no registrados tras verificar un captcha, permitiendo el acceso anónimo.
- **Cuerpo (Body):** CaptchaPayload
- **Respuesta:**
 - **200 OK:** Devuelve un SessionTokenPayload.
 - **400 Bad Request:** El captcha es inválido.
 - **500 Internal Server Error:** Error interno del servidor.

Iniciar Sesión de Usuario (Login)

- **Método:** POST
- **Ruta:** /api/auth/login
- **Acción:** Autentica a un usuario registrado con su email y contraseña.
- **Cuerpo (Body):** LoginPayload
- **Respuesta:**
 - **200 OK:** Devuelve un SessionTokenPayload.
 - **400 Bad Request:** Datos mal formados.
 - **401 Unauthorized:** Credenciales incorrectas.
 - **500 Internal Server Error:** Error interno del servidor.

Cerrar Sesión de Usuario (Logout)

- **Método:** POST
- **Ruta:** /api/auth/logout

- **Acción:** Invalida el token de sesión del usuario.
- **Autenticación:** Requerida (Bearer Token).
- **Respuesta:**
 - **204 No Content:** Sesión cerrada con éxito.
 - **401 Unauthorized:** Token no válido o ausente.

Registrar un Nuevo Usuario

- **Método:** POST
- **Ruta:** /api/users/register
- **Acción:** Crea un nuevo usuario pendiente de confirmación por correo electrónico.
- **Cuerpo (Body):** RegisterPayload
- **Respuesta:**
 - **201 Created:** Usuario creado, pendiente de verificación.
 - **400 Bad Request:** Datos inválidos (ej. contraseña débil, email mal formado).
 - **409 Conflict:** El email ya está en uso.
 - **500 Internal Server Error:** Error interno del servidor.

Verificar Correo Electrónico

- **Método:** POST
- **Ruta:** /api/auth/verify/{userId}
- **Acción:** Valida la cuenta de un usuario a través del enlace de verificación enviado a su correo.
- **Parámetros de Ruta:** userId (el identificador único del usuario).
- **Respuesta:**
 - **200 OK:** Usuario verificado correctamente.
 - **400 Bad Request:** Enlace inválido o expirado.
 - **404 Not Found:** El usuario no existe.
 - **500 Internal Server Error:** Error interno del servidor.

Eliminar Cuenta de Usuario

- **Método:** DELETE
- **Ruta:** /api/users/me
- **Acción:** Desactiva o elimina la cuenta del usuario actualmente autenticado.
- **Autenticación:** Requerida (Bearer Token).
- **Respuesta:**
 - **204 No Content:** Usuario eliminado con éxito.
 - **401 Unauthorized:** Token no válido o ausente.
 - **404 Not Found:** El usuario no existe.

Exportar Datos de Usuario

- **Método:** GET
- **Ruta:** /api/users/me/export
- **Acción:** Exporta toda la información asociada a la cuenta del usuario autenticado.
- **Autenticación:** Requerida (Bearer Token).

- **Respuesta:**
 - **200 OK:** Devuelve un JSON con todos los datos del usuario.
 - **401 Unauthorized:** Token no válido o ausente.

Endpoints de Chat

Endpoints para la gestión de conversaciones con la IA. Todos requieren autenticación.

Obtener Lista de Conversaciones

- **Método:** GET
- **Ruta:** /api/conversations
- **Acción:** Devuelve una lista con los títulos e identificadores de las conversaciones del usuario.
- **Autenticación:** Requerida.
- **Respuesta:**
 - **200 OK:** Devuelve un ConversationListPayload.
 - **401 Unauthorized:** Token no válido o ausente.

Obtener Conversación Completa

- **Método:** GET
- **Ruta:** /api/chats/{chatId}
- **Acción:** Devuelve el historial completo de mensajes de una conversación específica.
- **Autenticación:** Requerida.
- **Respuesta:**
 - **200 OK:** Devuelve un MessageListPayload.
 - **401 Unauthorized:** No autorizado.
 - **404 Not Found:** La conversación no existe o no pertenece al usuario.

Enviar Mensaje a una Conversación

- **Método:** PATCH
- **Ruta:** /api/chats/{chatId}
- **Acción:** Envía una pregunta del usuario a una conversación existente y devuelve la respuesta de la IA.
- **Autenticación:** Requerida.
- **Cuerpo (Body):** MessagePayload
- **Respuesta:**
 - **200 OK:** Devuelve un MessagePayload con la respuesta de la IA.
 - **400 Bad Request:** Mensaje mal formado.

Eliminar Conversación

- **Método:** DELETE
- **Ruta:** /api/chats/{chatId}
- **Acción:** Elimina una conversación específica.
- **Autenticación:** Requerida.

- **Respuesta:**
 - **204 No Content:** Conversación eliminada con éxito.
 - **404 Not Found:** La conversación no existe o no pertenece al usuario.

Importar Conversación

- **Método:** PUT
- **Ruta:** /api/chats/{chatId}
- **Acción:** Importa un historial de mensajes a una conversación.
- **Autenticación:** Requerida.
- **Cuerpo (Body):** MessageListPayload
- **Respuesta:**
 - **200 OK:** Importación exitosa.
 - **400 Bad Request:** Datos de importación inválidos.

Investigación y evaluación del uso de MCPs para web scraping con Puppeteer y Filesystem

Fecha del experimento: 09/04/2025

Introducción

En este experimento se comprobó si los Model-Controlled Programs (MCPs) son una solución viable para tareas de Web Scraping.

A continuación, se detallan los experimentos realizados, herramientas utilizadas, los resultados obtenidos y las limitaciones encontradas.

Entorno y herramientas

Para las pruebas se utilizaron un Cliente de Cursor AI y los MCP Puppeteer (para control de navegador) y "fs / filesystem" (para escritura en disco).

Pruebas

Primera prueba: Web Scraping funcional con MCPs

El *prompt* utilizado fue el siguiente:

Accede al plan de estudios de enfermería de la Universidad de León, hazme un resumen de sus asignaturas con sus guías de estudio detallando porcentajes de exámenes, prácticas y fechas de exámenes. Me lo guardas en un fichero llamado planEnfermeria.txt..

El proceso de *scraping* se ejecutó correctamente. El contenido fue extraído, estructurado y almacenado en el archivo planEnfermeria.txt.

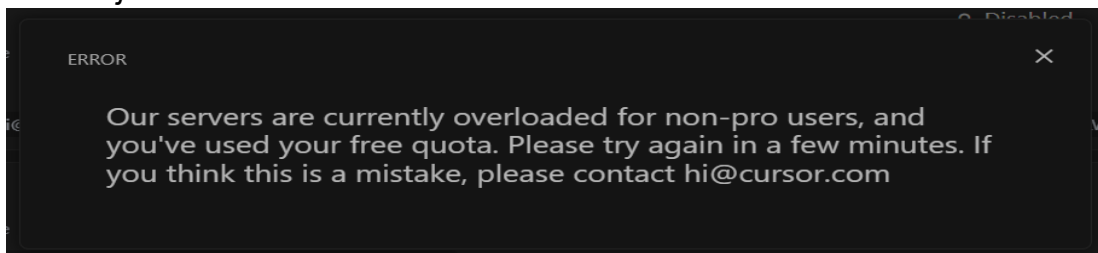
El resultado demuestra que MCP pueden ser eficaces para tareas básicas o moderadamente complejas de *scraping* en sitios estáticos o semi-dinámicos.

Segunda prueba: Fallo en tareas de *scraping* más complejas.

El *prompt* utilizado fue el siguiente:

Accede al sitio web de <https://www.ncsc.gov.uk/section/advice-guidance/all-articles?q=&defaultTypes=guidance,information,blog-post,collection&sort=date%2Bdesc> accede a todos los artículos guárdalos en un fichero llamado cyberNcscArticles2.txt. En el caso de que te quedes sin artículos, haces clic al botón de "Show 10 more". Cada artículo deberá de contener un Título, el contenido de este (haz un copia y pega), un Autor o referencia, la fecha del artículo y el enlace de este.

Salida de Ejecución:



Al ejecutar el mismo *prompt* utilizando otro modelo, como Claude, y manteniendo los mismos parámetros de configuración (MCP), se obtiene la siguiente salida:

Fichero	de	Errores:
# Artículos del NCSC	(National Cyber Security Centre)	

Fecha de recopilación: 07-04-2025
Este archivo debería contener artículos del sitio web <https://www.ncsc.gov.uk/>
Sin embargo, hubo problemas técnicos al acceder al sitio web.
Revisando en los logs de MCP de Puppeteer, encontré lo siguiente:
2025-04-01T15:40:10.584Z [puppeteer] [info] Message from server:
{ "jsonrpc": "2.0", "id": 70, "error": { "code": -32601, "message": "Method not found" } }>

Después de investigar las causas del error se cree que el sitio web tiene bloqueado esta forma de rastreo web.

Resultados

En conclusión, aunque esta tecnología resulta adecuada para la simulación de programas básicos de rastreo web, su uso como herramienta principal no es recomendada a causa de limitaciones relativas a la estabilidad y dependencia operativa que podría generar en el proyecto.

Participantes

Han tomado parte en el proyecto:

- Santiago Ares Alfayate
- César Bermejo Fernández
- Ander Chueca Rodríguez
- Gabriel Díez Tejedor
- Sergio González Rebollo
- Esteban Peña Salazar
- Álvaro Prieto Álvarez
- Carlos Prieto Viñuela

Conclusión

El presente informe tiene como objetivo proporcionar una visión detallada y estructurada del estado actual del proyecto, documentando de manera rigurosa el trabajo realizado hasta la fecha de emisión. Se centra especialmente en los aspectos relacionados con el desarrollo técnico y la gestión del proyecto, incluyendo los avances alcanzados, las tareas completadas, las herramientas utilizadas, así como los retos identificados y las estrategias adoptadas para su resolución. Esta documentación permite evaluar el grado de cumplimiento de los objetivos establecidos, facilita la toma de decisiones informadas para las siguientes fases y garantiza la trazabilidad del progreso del proyecto.

Situación actual

Respecto al propio proyecto se ha desarrollado por completo varios scripts para la extracción de datos, los cuales se guardan en una base de datos orientada a grafos, se implementó también una inteligencia artificial para la clasificación de las temáticas de los datos extraídos, el desarrollo de la gestión de documentos preprocesados, una API-Restful para la gestión y registro de usuarios los cuales se guardan en una base de datos relacional

Todo el ecosistema del proyecto está siendo implementado mediante un repositorio de GitHub y desplegado mediante contenedores virtualizados.

Actualmente, el equipo está trabajando en el desarrollo de un sistema RAG (Retrieval-Augmented Generation) y del despliegue del sistema en la infraestructura del centro de supercomputación.